量化

Quantization

原文作者：@techNmak。

先列专业术语对照表，再提供逐页完整中文译文。每个原文页面的译文之后附原文矢量对照页，完整保留公式、图表、代码和英文引用，便于核对。

正文中文：LXGW WenKai（霞鹜文楷）；英文与数字：Times New Roman。原文对照页保留原始字体。

本文采用翻译辅助服务生成初稿，统一技术术语并检查结构、字体和导航；涉及精确数值、公式或实现细节时，可直接对照相邻原文页。

专业术语翻译对照表

英文原词	中文译名	缩写／符号
Quantization	量化	
Inference	推理	
Asymmetric Quantization	非对称量化	
Activation Quantization	激活量化	
Transformer Block	Transformer 块	
Mixed Precision	混合精度	
Representation	表示	
Mental Model	理解框架	
Transformer	Transformer 模型	Transformer
Per-channel	逐通道	
Calibration	校准	
Activations	激活	
Activation	激活	
Zero-point	零点	
Per-tensor	逐张量	
Group Size	组大小	
Benchmark	基准测试	
Precision	精度	
Quantized	量化后的	
Per-group	逐组	
Training	训练	
KV Cache	键值缓存	KV Cache
Clipping	截断	
Rounding	舍入	
Codebook	码本	
Matrices	矩阵	
Latency	延迟	
Packing	打包	
Vectors	向量	
Kernels	内核	
Tensor	张量	
Kernel	内核	

英文原词	中文译名	缩写／符号
Value	值	V
Model	模型	
Gradient	梯度	
Values	值	
Token	词元	
Post-training Quantization	训练后量化	PTQ
Floating-point	浮点	
Floating Point	浮点	
Dynamic Range	动态范围	
Scaling	缩放	
Index	索引	
Quantization Scale	量化尺度	
Dequantization	反量化	
Symmetric Quantization	对称量化	
Quantization Error	量化误差	
Outlier	离群值	
Low-bit	低比特	
Accuracy	准确性	
Models	模型	
Keys	键	
Matrix	矩阵	
Weight-only Quantization	仅权重量化	
Matrix Multiplication	矩阵乘法	
Head Dimension	头维度	
Accumulator	累加器	
Attention	注意力机制	Attention
KV Heads	键值头	
Synchronization	同步	
Performance	性能	
Core	核心	
Quantization-aware Training	量化感知训练	QAT
Forward Pass	前向传播	
Fine-tuning	微调	
Pipeline	流水线	

英文原词	中文译名	缩写／符号
Memory Traffic	内存流量	
Tensor Shape	张量形状	
Matmul	矩阵乘法	
Memory Footprint	内存占用	
Bandwidth-bound	带宽受限	
Throughput	吞吐量	
Combine	合并	
Normalization	归一化	
Accumulation	累加	
Embeddings	嵌入向量	
FP32	32 位浮点	FP32
Vector	向量	
Global Memory	全局内存	
Operators	算子	
Vocabulary	词表	
Layer Normalization	层归一化	LayerNorm
Softmax	Softmax 归一化	Softmax
Operator	算子	
Cache Payload	缓存有效载荷	
Device Memory	设备内存	
Key	键	K
Tokens	词元	
Materialization	物化	
Serving	推理服务	
Allocator Overhead	分配器开销	
Effective Bits	有效位数	
GEMM	通用矩阵乘法	GEMM
Launch Overhead	启动开销	
Tensor Cores	张量核心	
Batch Size	批大小	
Resident	驻留	
Output Heads	输出头	
Generation	生成	

目录导航

点击章节标题跳转到中文译文，点击“原文”核对公式和图表。页码按本译本最终分页生成。

封面与说明 · 第 1 页

专业术语翻译对照表 · 第 2 页

量化（Quantization） · 第 7 页 ［原文 1］

- 方向 · 第 9 页 ［原文 2］

1 - 量化（Quantization） 是什么 · 第 11 页 ［原文 3］

2 - 低 精度（Precision） 不是一回事 · 第 13 页 ［原文 4］

3 - 仿射整数 量化（Quantization） · 第 15 页 ［原文 5］

4 - 对称与不对称 · 第 17 页 ［原文 6］

5 - 工作示例 I：对称 INT8 · 第 19 页 ［原文 7］

6 - 工作示例 II：不对称 UINT8 · 第 21 页 ［原文 8］

7 - 舍入（Rounding）、截断（Clipping） 和错误 · 第 23 页 ［原文 9］

8 - 量化（Quantization） 粒度 · 第 25 页 ［原文 10］

9 - 工作示例 III：粒度 · 第 27 页 ［原文 11］

10 - 量化后的（Quantized） 是什么？ · 第 29 页 ［原文 12］

11 - 校准（Calibration） 和观察员 · 第 31 页 ［原文 13］

12 - 静态 VS 动态 PTQ · 第 33 页 ［原文 14］

13 - 量化感知训练（Quantization-aware Training） · 第 35 页 ［原文 15］

14 - 仅整数与去量化执行 · 第 37 页 ［原文 16］

15 - 仅权重量化（Weight-only Quantization） · 第 39 页 ［原文 17］

16 - 为什么“4 位 模型（Model）”不是 4 位计算 · 第 41 页 ［原文 18］

17 - 激活（Activation） 异常值 · 第 43 页 ［原文 19］

18 - LLM.INT8() · 第 45 页 ［原文 20］

19 - 平滑量化 · 第 47 页 ［原文 21］

20 - 零定量 · 第 49 页 ［原文 22］

21 - GPTQ · 第 51 页 ［原文 23］

22 - 预警 · 第 53 页 ［原文 24］

23 - GPTQ、AWQ、SMOOTHQUANT、LLM.INT8 · 第 55 页 ［原文 25］

24 - NF4 · 第 57 页 ［原文 26］

25 - QLORA 和双 量化（Quantization） · 第 59 页 ［原文 27］

26 - FP8：E4M3 和 E5M2 · 第 61 页 ［原文 28］

27 - FP8 缩放（Scaling） 和 混合精度（Mixed Precision） · 第 63 页 ［原文 29］

28 - 块级 FP8 和 FP4 · 第 65 页 ［原文 30］

29 - KV 缓存 量化（Quantization） · 第 67 页 ［原文 31］

30 - 基维 · 第 69 页 ［原文 32］

31 - 科维量子 · 第 71 页 ［原文 33］

32 - 内存核算 · 第 73 页 ［原文 34］

33 - 工作记忆示例：组元数据 · 第 75 页 ［原文 35］

34 - 内核（Kernels） 和硬件 · 第 77 页 ［原文 36］

35 - 延迟（Latency）、吞吐量（Throughput） 和带宽 · 第 79 页 ［原文 37］

36 - 透明的 Pytorch 量化器 · 第 81 页 ［原文 38］

37 - 常见的实施错误 · 第 83 页 ［原文 39］

38 - 完整的 理解框架（Mental Model） · 第 85 页 ［原文 40］

参考 · 第 87 页 ［原文 41］

量化（Quantization）

对应原文第 1 页；点击查看相邻原文对照页

了解 量化（Quantization） 从标度和零点到 INT4、FP8 和现代 LLM 推理（Inference） 面向 AI 工程师、开发人员和认真学习者的简短技术手册 手册 10 - 了解 AI 系列

Understanding Quantization

From Scales and Zero-Points to INT4, FP8, and Modern LLM Inference

A short technical handbook for AI engineers, developers, and serious learners

@techNmak

Handbook 10 - Understanding AI Series

- 方向

对应原文第 2 页；点击查看相邻原文对照页

- 方向 本手册将阐明的内容 量化（Quantization） 通常被概括为“使用更少的位”。这在方向上是正确的，但在技术上是不完整的。一个有用的解释必须说明什么是 张量（Tensor） 是 量化后的（Quantized）、使用什么数值 码本（Codebook）、如何选择比例参数、舍入或剪裁什么、哪些操作实际上以较低的 精度（Precision） 运行，以及哪些成本仍保持在较高的 精度（Precision） 上。

到最后，您应该能够解释：

- 仿射量化/反量化方程以及尺度和零点（Zero-point） 的作用；

- 对称与非对称量化（Asymmetric Quantization） 和逐张量（Per-tensor） 与逐通道（Per-channel）/逐组（Per-group） 粒度；

- 为什么舍入（Rounding）、截断（Clipping）、校准（Calibration） 和异常值会产生不同的错误模式；

- 仅权重、权重激活（Activation） 和KV 缓存量化（Quantization） 作为不同的部署问题；

- PTQ、QAT、假量化（Quantization） 以及动态与静态激活量化（Activation Quantization）；

- 为什么“4 位模型（Model）”并不意味着每个值（Value） 或每个算术运算都是4 位；

- LLM.int8()、SmoothQuant、GPTQ、AWQ、NF4/QLoRA、FP8、KIVI 和 KVQuant 如何攻击不同的瓶颈；

- 为什么较低的位宽可以减少内存而不自动减少实际运行延迟（Latency）；

- 硬件内核（Kernels）、打包（Packing）、组大小（Group Size）、标度和混合精度（Mixed Precision）如何确定实际系统结果。

先决条件 熟悉 向量（Vectors）、矩阵（Matrices）、Transformer 模型（Transformer） 块以及重量、激活（Activations） 和 键值缓存（KV Cache） 之间的差异就足够了。早期关于 键值缓存（KV Cache） 和 Transformer 块（Transformer Block） 的手册是有用的上下文。

范围 本手册是关于数值 表示（Representation） 和执行的。它不会成为 CUDA 内核（Kernel） 手册、完整的混合 精度（Precision） 训练（Training） 指南或 基准测试（Benchmark） 目录。仅在更改 理解框架（Mental Model） 时才包含特定于硬件的行为。

了解 量化（Quantization） 研究锚点：[1]-[17]

- ORIENTATION

What this handbook will make clear

Quantization is often summarized as “use fewer bits.” That is directionally true and technically incomplete. A useful explanation has to say what tensor is quantized, what numerical codebook is used, how scale parameters are chosen, what is rounded or clipped, which operations actually run at low precision, and which costs remain at higher precision.

By the end, you should be able to explain:

- the affine quantize/dequantize equations and the roles of scale and zero-point;
- symmetric vs asymmetric quantization and per-tensor vs per-channel/per-group granularity;
- why rounding, clipping, calibration, and outliers create different error modes;
- weight-only, weight-activation, and KV-cache quantization as different deployment problems;
- PTQ, QAT, fake quantization, and dynamic vs static activation quantization;
- why “4-bit model” does not mean every value or every arithmetic operation is 4-bit;
- how LLM.int8(), SmoothQuant, GPTQ, AWQ, NF4/QLoRA, FP8, KIVI, and KVQuant attack different bottlenecks;
- why lower bit-width can reduce memory without automatically reducing wall-clock latency;
- how hardware kernels, packing, group size, scales, and mixed precision determine the real system outcome.

Prerequisites

Comfort with vectors, matrices, Transformer blocks, and the difference between weights, activations, and KV cache is enough. The earlier handbooks on KV cache and the Transformer block are useful context.

Scope

This handbook is about numerical representation and execution. It does not become a CUDA kernel manual, a complete mixed-precision training guide, or a benchmark catalog. Hardware-specific behavior is included only where it changes the mental model.

Research anchors: [1]-[17]

1 - 量化（Quantization） 是什么

对应原文第 3 页；点击查看相邻原文对照页

1 - 量化（Quantization） 是什么 用较小的可表示集替换较大的数值集 模型（Model） 通常以某种有限数值格式存储或计算 值（Values）。 量化（Quantization） 将这些 值（Values） 映射到较小的一组可表示级别。一个简单的标量量化器可以写成

其中 q 是存储的低 精度（Precision） 代码，x 是为以后计算而重构的 值（Value）。

对于神经网络，x 可能是权重、激活（Activation）、梯度（Gradient）、优化器状态或 KV 缓存元素。这些对象具有不同的分布和对错误的不同敏感度，因此“模型（Model） 是 量化后的（Quantized）”并不是完整的规范。

因此，一个有用的部署描述至少需要四个部分：

1. 目标：权重、激活（Activations）、键值缓存（KV Cache） 或另一个 张量（Tensor）；

2. 格式：INT8、INT4、FP8、NF4、FP4 或其他 码本（Codebook）；

3.粒度：张量（Tensor）、通道、词元（Token）、组或块的一个尺度；

4.执行：实际的内核（Kernel）如何消耗打包的值（Values）。

量化（Quantization） 在普通情况下是有损的，因为多个高 精度（Precision） 值（Values） 映射到相同的代码。工程目标不是准确保留每个数字。重要的是保留 模型（Model） 行为，同时减少内存、带宽或计算成本。

理解框架（Mental Model）。位宽只是 量化（Quantization） 方案的一维。映射、尺度粒度、校准（Calibration） 规则和 内核（Kernel） 同样重要。

研究锚点：[1]、[2] 理解 量化（Quantization）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

1 - WHAT QUANTIZATION IS

Replace a large numerical set with a smaller representable set

A model normally stores or computes values in some finite numerical format. Quantization maps those values to a smaller set of representable levels. A simple scalar quantizer can be written as

$$q = Q(x), \qquad \hat{x} = D(q),$$

where q is the stored low-precision code and $\hat{x}$ is the value reconstructed for later computation.

For a neural network, x might be a weight, activation, gradient, optimizer state, or KV-cache element. These objects have different distributions and different sensitivity to error, so "the model is quantized" is not a complete specification.

A useful deployment description therefore needs at least four pieces:

1. **target:** weights, activations, KV cache, or another tensor;
2. **format:** INT8, INT4, FP8, NF4, FP4, or another codebook;
3. **granularity:** one scale for a tensor, channel, token, group, or block;
4. **execution:** how packed values are consumed by the actual kernel.

Quantization is lossy in the ordinary case because multiple high-precision values map to the same code. The engineering goal is not to preserve every number exactly. It is to preserve the model behavior that matters while reducing memory, bandwidth, or compute cost.

Mental model. Bit-width is only one dimension of a quantization scheme. The mapping, scale granularity, calibration rule, and kernel are equally important.

Research anchors: [1], [2]

2 - 低 精度（Precision） 不是一回事

对应原文第 4 页；点击查看相邻原文对照页

2 - 低 精度（Precision） 不是一回事 整数 量化（Quantization）、浮点（Floating-point） 格式和代码本不同 短语“8 位”可以描述数字上非常不同的表示形式。

示例 表示（Representation） idea 值（Values） INT8 仿射整数代码加标度/零点（Zero-point） 在固定标度 FP8 符号下均匀的间距，指数、尾数非均匀； 动态范围（Dynamic Range）来了

NF4 非均匀分位数样式级别 BF16/FP16 减少了 浮点（Floating Point） 浮点（Floating-point） 间距，通常被视为 混合精度（Mixed Precision） 而不是 PTQ 这种区别很重要，因为两个“8 位”方法可以具有不同的动态范围（Dynamic Range）、错误分布、元数据、硬件支持和算术路径。

BF16 和 FP16 通常被讨论为简化的 精度（Precision） 浮点（Floating-point） 格式，而不是 训练后量化（Post-training Quantization） 方案。 FP8 更接近边界：现代 训练（Training）/推理（Inference） 系统在 FP8 张量周围使用 缩放（Scaling） 配方，因此经常在与 量化（Quantization） 相同的低 精度（Precision） 工具箱中讨论它。

同样，NF4 也不仅仅是“不同尺度的 INT4”。它是一个 4 位 码本（Codebook），围绕 QLoRA 中的标准化正常权重假设而设计。当前的 bitsandbytes 代码明确将 NF4 视为查找 码本（Codebook），而不是符号/指数/尾数浮点格式。

重要：切勿仅从位宽推断格式的数字行为。“4 位”或“8 位”不足以重构量化器。

研究锚点：[8]、[9]、[14]、[16] 将 量化（Quantization） 4 位 索引（Index） 理解为固定的非均匀 码本（Codebook）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

2 - LOW PRECISION IS NOT ONE THING

Integer quantization, floating-point formats, and codebooks differ

The phrase “8-bit” can describe numerically very different representations.

Example	Representation idea	Spacing of values
INT8 affine	integer code plus scale/zero-point	uniform under a fixed scale
FP8	sign, exponent, mantissa	non-uniform; dynamic range comes from exponent
NF4	4-bit index into a fixed non-uniform codebook	non-uniform quantile-style levels
BF16/FP16	reduced floating point	floating-point spacing, usually treated as mixed precision rather than PTQ

This distinction matters because two “8-bit” methods can have different dynamic range, error distribution, metadata, hardware support, and arithmetic paths.

BF16 and FP16 are usually discussed as reduced-precision floating-point formats rather than post-training quantization schemes. FP8 sits closer to the boundary: modern training/inference systems use scaling recipes around FP8 tensors, so it is often discussed in the same low-precision toolbox as quantization.

Likewise, NF4 is not simply “INT4 with a different scale.” It is a 4-bit codebook designed around a normalized normal-weight assumption in QLoRA. Current bitsandbytes code explicitly treats NF4 as a lookup codebook rather than a sign/exponent/mantissa float format.

Important. Never infer the numerical behavior of a format from bit-width alone. “4-bit” or “8-bit” is not enough to reconstruct the quantizer.

Research anchors: [8], [9], [14], [16]

3 - 仿射整数 量化（Quantization）

对应原文第 5 页；点击查看相邻原文对照页

3 - AFFINE INTEGER 量化尺度（Quantization Scale） 加 零点（Zero-point） 将实数 值（Values） 映射到整数代码 常见的均匀仿射量化器表示实数 值（Value） 大约为

其中 s > 0 是比例，z 是整数 零点（Zero-point），q 是 [qmin, qmax] 中的整数代码。

量化（Quantization） 和 反量化（Dequantization） 则 q = 钳位

圆形

+ z、q 最小值、q 最大值

比例尺确定相邻可表示实数 值（Values） 之间的距离。 零点（Zero-point） 确定哪个整数代码代表实零。 舍入（Rounding） 产生有限的 精度（Precision）；夹紧手柄 值（Values） 超出所选的可表示范围。

对于非对称最小/最大映射，常见的起点是 s = rmax- rmin qmax- qmin

z 四舍五入并剪裁到整数范围。

确切的约定因运行时而异。一些对称 INT8 实现有意使用 [-127, 127] 而不是完整的有符号的 [-128, 127] 范围，以保持零周围的精确对称性。

理解框架（Mental Model）。仿射 量化（Quantization） 是实值区间和有限整数网格之间的坐标变换。

研究锚点：[1]、[2]、[13] 理解 量化（Quantization）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

3 - AFFINE INTEGER QUANTIZATION

Scale plus zero-point map real values to integer codes

A common uniform affine quantizer represents a real value approximately as

$$x \approx s(q - z),$$

where $s > 0$ is a scale, z is an integer zero-point, and q is an integer code in $[q_{\min}, q_{\max}]$.

Quantization and dequantization are then

$$q = \text{clamp}\Big(\text{round}\Big(\frac{x}{s}\Big) + z,\ q_{\min}, q_{\max}\Big),$$

$$\hat{x} = s(q - z).$$

The scale determines the distance between adjacent representable real values. The zero-point determines which integer code represents real zero. Rounding produces finite precision; clamping handles values outside the chosen representable range.

For an asymmetric min/max mapping, a common starting point is

$$s = \frac{r_{\max} - r_{\min}}{q_{\max} - q_{\min}}, \qquad z \approx q_{\min} - \frac{r_{\min}}{s},$$

with z rounded and clipped to the integer range.

The exact conventions vary by runtime. Some symmetric INT8 implementations intentionally use $[-127, 127]$ instead of the full signed $[-128, 127]$ range to preserve exact symmetry around zero.

Mental model. Affine quantization is a coordinate transform between a real-valued interval and a finite integer grid.

Research anchors: [1], [2], [13]

4 - 对称与不对称

对应原文第 6 页；点击查看相邻原文对照页

4 - 对称 VS 不对称 我们需要学习 零点（Zero-point） 吗？

在 对称量化（Symmetric Quantization） 中，实际范围以零为中心。对于有符号整数，零点（Zero-point）通常固定为零，并且比例由最大绝对量级确定：

s = 最大值|x|最大q值

这很简单，并且对于分布近似居中的权重来说通常很有吸引力。

在 非对称量化（Asymmetric Quantization） 中，观察到的最小值和最大值可以映射到整数范围的不同端。 零点（Zero-point） 移动网格，以便仍然可以表示真正的零。当 张量（Tensor） 强烈偏离零时，这可以更有效地使用可用代码范围。

对称 非对称 零点（Zero-point） 通常固定为从范围派生的 0 最佳直觉中心范围 移位范围 元数据更简单，包括非零 z 常见的许多权重方案 激活（Activations） / KV 方案（其中偏移量很重要） 没有普遍规则表明权重必须对称或 激活（Activations） 不对称。选择取决于分布、后端和 量化（Quantization） 方法。

重要：非零 零点（Zero-point） 不是“偏差”。它是数字坐标系的一部分，用于将整数代码解码回近似实数值（Values）。

研究锚点：[1]、[2]、[13] 理解 量化（Quantization）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

4 - SYMMETRIC VS ASYMMETRIC

Do we need a learned zero-point?

In **symmetric** quantization, the real range is centered around zero. For signed integers, the zero-point is usually fixed at zero and the scale is determined by the largest absolute magnitude:

$$s = \frac{\max |x|}{q_{\max}}, \qquad z = 0.$$

This is simple and often attractive for weights whose distributions are approximately centered.

In **asymmetric** quantization, the observed minimum and maximum can map to different ends of the integer range. The zero-point shifts the grid so that real zero is still representable. This can use the available code range more efficiently when the tensor is strongly offset from zero.

	Symmetric	**Asymmetric**
Zero-point	typically fixed at 0	derived from range
Best intuition	centered range	shifted range
Metadata	simpler	includes nonzero z
Common use	many weight schemes	activations / KV schemes where offset matters

There is no universal rule that weights must be symmetric or activations asymmetric. The choice depends on the distribution, backend, and quantization method.

Important. A nonzero zero-point is not "bias." It is part of the numerical coordinate system used to decode integer codes back into approximate real values.

Research anchors: [1], [2], [13]

5 - 工作示例 I：对称 INT8

对应原文第 7 页；点击查看相邻原文对照页

5 - 工作示例 I：对称 INT8 量化、存储，然后重建

并使用带符号的对称 INT8 和代码[-127, 127]。

最大星等为1.2，所以

当 z = 0 时， q = round(x/s) = [-127,-53, 0, 32, 106]。

反量化（Dequantization） 给出

在这个小例子中可以看到三件事。首先，当从最大幅度导出尺度时，可以几乎准确地表示端点。其次，内部 值（Values） 移动到最近的网格点。第三，通过舍入到最接近的值，未裁剪的标量最多被替换大约半个 量化（Quantization） 步长（除了打破平局约定之外）。

如果 离群值（Outlier） 迫使比例变得更大，则对于共享该比例的每个较小的 值（Value），网格将变得更粗糙。这就是粒度和 离群值（Outlier） 处理在 LLM 量化（Quantization） 中如此重要的原因之一。

理解框架（Mental Model）。 量化误差（Quantization Error） 并不神秘：它是原始 值（Value） 与最近允许的重建级别之间的距离，加上任何 截断（Clipping） 误差。

研究锚点：[1]、[2] 理解 量化（Quantization） 因此，重建误差 (x_hat - x) 近似为

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

5 - WORKED EXAMPLE I: SYMMETRIC INT8

Quantize, store, then reconstruct

Take

$$x = [-1.2, -0.5, 0, 0.3, 1.0]$$

and use signed symmetric INT8 with codes $[-127, 127]$.

The largest magnitude is 1.2, so

$$s = \frac{1.2}{127} \approx 0.00944882.$$

With $z = 0$,

$$q = \text{round}(x/s) = [-127, -53, 0, 32, 106].$$

Dequantization gives

$$\hat{x} = sq \approx [-1.2000, -0.5008, 0, 0.3024, 1.0016].$$

The reconstruction error (x_hat - x) is therefore approximately

$$[0, -0.0008, 0, +0.0024, +0.0016].$$

Three things are visible in this tiny example. First, the endpoints can be represented almost exactly when the scale is derived from the maximum magnitude. Second, interior values move to the nearest grid point. Third, with round-to-nearest, an unclipped scalar is displaced by at most roughly half one quantization step (apart from tie-breaking conventions).

If an outlier forced the scale to become much larger, the grid would become coarser for every smaller value sharing that scale. That is one reason granularity and outlier handling matter so much in LLM quantization.

Mental model. Quantization error is not mysterious: it is the distance between the original value and the nearest allowed reconstruction level, plus any clipping error.

Research anchors: [1], [2]

6 - 工作示例 II：不对称 UINT8

对应原文第 8 页；点击查看相邻原文对照页

6 - 工作示例 II：不对称 UINT8 移位范围使用非零 零点（Zero-point）

并将观察到的范围映射到 UINT8 代码[0, 255]。

范围宽度为2.0，所以

零点（Zero-point） 是

量化（Quantization） 给出 q = round(x/s + 51) = [0, 82, 173, 255]。

重建给出

非零 零点（Zero-point） 让整数代码 51 代表实数零，即使观察到的范围不是围绕零对称的。这就是不对称仿射 量化（Quantization） 存在的核心原因。

运行时可以使用 校准（Calibration） 统计信息选择一个范围，而不是精确的最小值和最大值。 截断（Clipping） 少量的极端 值（Values） 有时可以减少总误差，因为它允许在大多数 值（Values） 居住的区域上形成更精细的网格。

重要：“使用最小值和最大值”是一个简单的 校准（Calibration） 规则，而不是法律。实用的量化器可以优化 截断（Clipping） 阈值或使用直方图、百分位数、学习范围或特定于方法的目标。

研究锚点：[1]、[12]、[13] 理解 量化（Quantization）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

6 - WORKED EXAMPLE II: ASYMMETRIC UINT8

A shifted range uses a nonzero zero-point

Take

$$x = [-0.4, 0.24, 0.96, 1.6]$$

and map the observed range to UINT8 codes $[0, 255]$.

The range width is 2.0, so

$$s = \frac{1.6 - (-0.4)}{255} = \frac{2}{255} \approx 0.00784314.$$

The zero-point is

$$z \approx 0 - \frac{-0.4}{s} = 51.$$

Quantization gives

$$q = \text{round}(x/s + 51) = [0, 82, 173, 255].$$

Reconstruction gives

$$\hat{x} = s(q - 51) \approx [-0.4000, 0.2431, 0.9569, 1.6000].$$

The nonzero zero-point lets integer code 51 represent real zero even though the observed range is not symmetric around zero. That is the central reason asymmetric affine quantization exists.

A runtime may choose a range using calibration statistics rather than the exact minimum and maximum. Clipping a small number of extreme values can sometimes reduce total error because it allows a finer grid over the region where most values live.

Important. "Use min and max" is a simple calibration rule, not a law. Practical quantizers may optimize clipping thresholds or use histograms, percentiles, learned ranges, or method-specific objectives.

Research anchors: [1], [12], [13]

7 - 舍入（Rounding）、截断（Clipping） 和错误

对应原文第 9 页；点击查看相邻原文对照页

7 - 舍入（Rounding）、截断（Clipping） 和错误 三种不同的失真源 量化误差（Quantization Error） 通常被视为一件事进行讨论，但至少应将三种机制分开。

舍入（Rounding） 错误。可表示区间内的 值（Value） 被移动到最近的网格/码本（Codebook） 级别。

截断（Clipping） 错误。超出选定范围的 值（Value） 将饱和至端点。这可能比 量化（Quantization） 步长大得多。

尺度共享误差。一组异构的 值（Values） 共享一个秤。大的 离群值（Outlier） 可以强制许多其他小的 值（Values） 采用粗步长。

这就产生了一个核心的权衡。更宽的可表示范围会减少 截断（Clipping），但会增加级别之间的间距。较窄的范围可提高常见 值（Values） 的分辨率，但会限制更极端的情况。

对于统一的 量化（Quantization），近似步长与代码的 $\Delta \propto$ 范围宽度数量成正比。

位宽减半可显着减少级别数量：8 位可提供 256 个代码，而 4 位仅可提供 16 个代码。因此，在低位宽时，尺度选择和分布形状更为重要。

GPTQ、AWQ、SmoothQuant 和 OmniQuant 等方法的部分不同之处在于它们如何减少这些错误的下游影响，而不是仅仅在各处应用朴素的舍入到最近值。

理解框架（Mental Model）。 低比特（Low-bit） 量化（Quantization） 是一个错误分配问题：决定 精度（Precision） 哪里重要以及哪里可以容忍近似。

研究锚点：[5]、[6]、[7]、[12] 理解 量化（Quantization）

7 - ROUNDING, CLIPPING, AND ERROR

Three different sources of distortion

Quantization error is often discussed as one thing, but at least three mechanisms should be separated.

Rounding error. A value inside the representable interval is moved to the nearest grid/codebook level.

Clipping error. A value outside the selected range is saturated to an endpoint. This can be much larger than one quantization step.

Scale-sharing error. A group of heterogeneous values shares one scale. A large outlier can force a coarse step size for many otherwise small values.

This creates a central trade-off. A wider representable range reduces clipping but increases the spacing between levels. A narrower range improves resolution for common values but clips more extremes.

For uniform quantization, the approximate step size is proportional to

$$\Delta \propto \frac{\text{range width}}{\text{number of codes}}.$$

Halving bit-width dramatically reduces the number of levels: 8 bits gives 256 codes, while 4 bits gives only 16. At low bit-width, scale selection and distribution shape therefore matter much more.

Methods such as GPTQ, AWQ, SmoothQuant, and OmniQuant differ partly in how they reduce the downstream effect of these errors rather than merely applying naive round-to-nearest everywhere.

Mental model. Low-bit quantization is an error-allocation problem: decide where precision matters and where approximation can be tolerated.

Research anchors: [5], [6], [7], [12]

8 - 量化（Quantization） 粒度

对应原文第 10 页；点击查看相邻原文对照页

8 - 量化（Quantization） 粒度 一台秤可容纳多少个 值（Values）？

量化器可以在不同范围内共享参数。

粒度 一个尺度涵盖权衡 逐张量（Per-tensor） 整个 张量（Tensor） 元数据最少，适应性最弱 逐通道（Per-channel） 一个通道/行/列更适合，更多尺度 逐组（Per-group） 固定大小的权重组 低比特（Low-bit） LLM 权重 Per-词元（Token） 值（Values） 关联具有一个 词元（Token） 适应 词元（Token） 变化的 激活（Activations）/缓存 每块硬件定义的块可实现本地 缩放（Scaling） 和硬件友好的格式 轴很重要。除非我们说明哪个维度定义了通道，否则“逐通道（Per-channel）”是不完整的。

权重 矩阵（Matrices） 可以根据方法/内核（Kernel） 沿输入或输出维度进行分组。

KV-cache方法可以为键（Keys）和值（Values）选择不同的粒度。

更细的粒度通常会减少 量化误差（Quantization Error），因为每个尺度覆盖更均匀的子集。但它也会增加规模元数据，并使 打包（Packing）、矢量化或 内核（Kernel） 设计变得复杂。

这就是为什么 组大小（Group Size） 不仅仅是 准确性（Accuracy） 旋钮。它也是一个系统参数。

重要：两个均描述为“INT4”的 模型（Models） 可能表现不同，因为一个使用 32 个权重的 逐组（Per-group） 秤，另一个使用 128 个权重，或者因为一个存储零点，而另一个是对称的。

研究锚点：[2]、[5]、[6]、[10]、[11]、[14]-[16] 理解 量化（Quantization）

8 - QUANTIZATION GRANULARITY

One scale for how many values?

A quantizer can share parameters across different scopes.

Granularity	One scale covers	Trade-off
Per-tensor	entire tensor	least metadata, weakest adaptation
Per-channel	one channel/row/column	better fit, more scales
Per-group	fixed-size group of weights	common compromise in low-bit LLM weights
Per-token	values associated with one token	adapts to token-varying activations/cache
Per-block	hardware-defined block	enables local scaling and hardware-friendly formats

The axis matters. “Per-channel” is incomplete unless we say which dimension defines the channel. Weight matrices can be grouped along input or output dimensions depending on the method/kernel. KV-cache methods can choose different granularities for keys and values.

Finer granularity usually reduces quantization error because each scale covers a more homogeneous subset. But it also increases scale metadata and can complicate packing, vectorization, or kernel design.

This is why group size is not just an accuracy knob. It is also a systems parameter.

Important. Two models both described as “INT4” can behave differently because one uses per-group scales of 32 weights and another uses 128, or because one stores zero-points while the other is symmetric.

Research anchors: [2], [5], [6], [10], [11], [14]-[16]

9 - 工作示例 III：粒度

对应原文第 11 页；点击查看相邻原文对照页

9 - 工作示例 III：粒度 逐通道（Per-channel） 秤可以保护小通道免受大通道的影响 考虑使用两排重量 矩阵（Matrix）

整个 张量（Tensor） 具有一个对称的 INT8 刻度，

第一行大约重构为

其相对较小的 值（Values） 被迫进入由第二行中的 12 确定的网格。

每行一个刻度，

第一行大约变成

对于这个玩具 矩阵（Matrix），均方重建误差从大约

确切的数字并不是关于 逐通道（Per-channel） 量化（Quantization） 的普遍说法。它们展示了一种机制：分离组可防止一个高动态范围通道设置另一个通道的 精度（Precision）。

理解框架（Mental Model）。更细的粒度需要花费更多的元数据来购买本地更好的分辨率。

研究锚点：[1]、[2]、[13] 理解 量化（Quantization）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

9 - WORKED EXAMPLE III: GRANULARITY

Per-channel scales can protect small channels from large ones

Consider a two-row weight matrix

$$W = \begin{bmatrix} -0.9 & -0.2 & 0.7 \\ -12 & 3 & 8 \end{bmatrix}.$$

With one symmetric INT8 scale for the whole tensor,

$$s_{\text{tensor}} = 12/127 \approx 0.09449.$$

The first row is reconstructed approximately as

$$[-0.9449, -0.1890, 0.6614].$$

Its relatively small values are forced onto a grid determined by the 12 in the second row.

With one scale per row,

$$s_1 = 0.9/127 \approx 0.00709, \qquad s_2 = 12/127 \approx 0.09449.$$

The first row becomes approximately

$$[-0.9000, -0.1984, 0.7016].$$

For this toy matrix, mean-squared reconstruction error drops from about

$$8.62 \times 10^{-4} \quad \text{to} \quad 2.59 \times 10^{-4}.$$

The exact numbers are not a universal claim about per-channel quantization. They show the mechanism: separating groups prevents one high-dynamic-range channel from setting the precision of another.

Mental model. Finer granularity spends more metadata to buy locally better resolution.

Research anchors: [1], [2], [13]

10 - 量化后的（Quantized） 是什么？

对应原文第 12 页；点击查看相邻原文对照页

10 - 量化后的（Quantized） 是什么？

权重、激活（Activations） 和 键值缓存（KV Cache） 具有不同的生命周期 三个重要的 LLM 目标很容易混淆。

权重是跨请求重用的 模型（Model） 参数。 仅权重量化（Weight-only Quantization） 主要减少 模型（Model） 存储和权重带宽。

激活是执行 模型（Model） 时产生的中间张量。它们的分布随着输入、层、词元（Token） 和执行阶段的变化而变化。 激活量化（Activation Quantization） 可以启用低 精度（Precision） 矩阵乘法（Matrix Multiplication），但它必须处理运行时变化和异常值。

在自回归 推理（Inference） 期间，KV 缓存是持久的每个请求 注意力机制（Attention） 状态。它的内存随着缓存的序列长度、批处理/并发、层、键值头（KV Heads）、头维度（Head Dimension） 和每个元素的字节数而增长。

权重 激活（Activations） 键值缓存（KV Cache） 模型（Model） 生命周期 操作生命周期 请求生命周期 W4A16 模型（Model）、W8A8 模型（Model） 和 FP8-KV 缓存 模型（Model） 解决不同的资源问题，即使它们都称为“量化后的（Quantized） LLM”。

重要：始终显式写入 量化（Quantization） 目标。 W4A16表示4位权重，16位激活（Activations）； 累加器（Accumulator） 精度（Precision） 由内核（Kernel） 单独定义。它与 W4A8 或 4 位 键值缓存（KV Cache） 不是同一系统。

研究锚点：[3]-[11]、[15]理解量化（Quantization）

10 - WHAT CAN BE QUANTIZED?

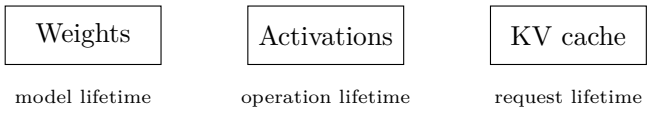
Weights, activations, and KV cache have different lifetimes

Three important LLM targets are easy to conflate.

Weights are model parameters reused across requests. Weight-only quantization primarily reduces model storage and weight bandwidth.

Activations are intermediate tensors produced while executing the model. Their distribution changes with input, layer, token, and execution phase. Activation quantization can enable low-precision matrix multiplication, but it must handle runtime variation and outliers.

KV cache is persistent per-request attention state during autoregressive inference. Its memory grows with cached sequence length, batch/concurrency, layers, KV heads, head dimension, and bytes per element.



A W4A16 model, a W8A8 model, and an FP8-KV-cache model solve different resource problems even if all are called “quantized LLMs.”

Important. Always write the quantization target explicitly. W4A16 means 4-bit weights with 16-bit activations; accumulator precision is separately defined by the kernel. It is not the same system as W4A8 or 4-bit KV cache.

Research anchors: [3]-[11], [15]

11 - 校准（Calibration） 和观察员

对应原文第 13 页；点击查看相邻原文对照页

11 - 校准（Calibration） 和观察员 在部署之前我们如何选择尺度？

训练后量化（Post-training Quantization） 通常需要代表性数据来估计范围、截断（Clipping） 阈值、激活（Activation） 统计数据或重建目标。这个过程通常称为校准。

一个简单的观察者可以跟踪跑步的最小值和最大值。基于直方图的观察者可以使用更详细的标准来估计分布并选择范围。 逐通道（Per-channel） 观察者收集不同通道的单独统计数据。

校准（Calibration） 数据不一定是原始 训练（Training） 数据。 GPTQ、AWQ、SmoothQuant 和 OmniQuant 都以不同的方式使用小型 校准（Calibration） 集/统计数据：

- GPTQ 在由近似二阶信息通知的重建过程中使用 校准（Calibration） 输入；

- AWQ 使用 激活（Activation） 统计数据来识别/保护显着的权重通道；

- SmoothQuant 使用 激活（Activation） 幅度来计算 激活（Activations） 和权重之间的等效重新缩放；

- OmniQuant 优化了截断（Clipping） 以及校准（Calibration） 样品上的等效转换。

较差的 校准（Calibration） 可能会产生技术上有效的 量化后的（Quantized） 模型（Model），其规模与实际工作负载不匹配。

理解框架（Mental Model）。 校准（Calibration） 不是“测试 准确性（Accuracy）”。它是根据代表性数值行为估计量化器参数的过程。

研究锚点：[5]-[7]、[12]、[13] 了解 量化（Quantization） PyTorch 的旧版 torch.ao.量化（Quantization） API 仍然公开最小值/最大值、移动平均值、逐通道（Per-channel） 和直方图观察器。当前量化（Quantization）开发集中在torchao； LLM 方法使用的确切 校准（Calibration） 策略可以更加专业化。

11 - CALIBRATION AND OBSERVERS

How do we choose scales before deployment?

Post-training quantization often needs representative data to estimate ranges, clipping thresholds, activation statistics, or reconstruction objectives. This process is usually called **calibration**.

A simple observer may track running minimum and maximum. Histogram-based observers can estimate a distribution and choose a range using a more elaborate criterion. Per-channel observers collect separate statistics for different channels.

PyTorch's legacy torch.ao.quantization APIs still expose min/max, moving-average, per-channel, and histogram observers. Current quantization development is centralized in torchao; the exact calibration strategy used by an LLM method can be more specialized.

Calibration data is not necessarily the original training data. GPTQ, AWQ, SmoothQuant, and OmniQuant all use small calibration sets/statistics in different ways:

- GPTQ uses calibration inputs in a reconstruction procedure informed by approximate second-order information;
- AWQ uses activation statistics to identify/protect salient weight channels;
- SmoothQuant uses activation magnitudes to compute an equivalent rescaling between activations and weights;
- OmniQuant optimizes clipping and equivalent transformations on calibration samples.

Poor calibration can produce a technically valid quantized model whose scales are mismatched to real workloads.

Mental model. Calibration is not "testing accuracy." It is the process of estimating quantizer parameters from representative numerical behavior.

Research anchors: [5]-[7], [12], [13]

12 - 静态 VS 动态 PTQ

对应原文第 14 页：点击查看相邻原文对照页

12 - 静态 VS 动态 PTQ 何时选择 激活量化（Activation Quantization） 参数?

经典的 量化（Quantization） 术语区分了两种常见的后 训练（Training） 模式。

静态 PTQ 使用 校准（Calibration） 传递在部署前确定 激活量化（Activation Quantization） 参数。在 推理（Inference） 中，运行时重用这些参数。

动态量化通常提前存储 量化后的（Quantized） 权重，但从当前运行时 张量（Tensor） 派生 激活（Activation） 比例。这会适应输入变化，但会增加运行时工作。

LLM文献有时使用不同的特定于方法的语言，因此这些术语不应机械地应用于每个现代内核（Kernel）。仅权重 GPTQ/AWQ 模型（Models）、FP8 缩放（Scaling） 配方和 KV 缓存量化器各自具有自己的缩放更新规则。

核心（Core） 系统权衡仍然有用：

预先计算的统计数据↔运行时适应。

运行时适应可以改进对当前 张量（Tensor） 的拟合，但如果 内核（Kernels） 不是围绕它们设计的，则缩放计算、归约运算、转换和 同步（Synchronization） 可能会消除 性能（Performance） 的部分优势。

重要：“动态”并不意味着权重本身被重新训练或改变。它指的是运行时值（Values） 的量化（Quantization） 参数何时确定。

研究锚点：[1]、[13]、[14] 理解 量化（Quantization）

12 - STATIC VS DYNAMIC PTQ

When are activation quantization parameters chosen?

Classical quantization terminology distinguishes two common post-training patterns.

Static PTQ uses a calibration pass to determine activation quantization parameters before deployment. At inference, the runtime reuses those parameters.

Dynamic quantization typically stores quantized weights ahead of time but derives activation scales from the current runtime tensor. This adapts to input variation but adds runtime work.

LLM literature sometimes uses different method-specific language, so these terms should not be mechanically applied to every modern kernel. Weight-only GPTQ/AWQ models, FP8 scaling recipes, and KV-cache quantizers each have their own scale-update rules.

The core systems trade-off is still useful:

$$\text{precomputed statistics} \quad \leftrightarrow \quad \text{runtime adaptation.}$$

Runtime adaptation can improve fit to the current tensor, but scale calculation, reduction operations, conversions, and synchronization can erase part of the performance benefit if kernels are not designed around them.

Important. “Dynamic” does not mean the weights themselves are retrained or changed. It refers to when quantization parameters for runtime values are determined.

Research anchors: [1], [13], [14]

13 - 量化感知训练（Quantization-aware Training）

对应原文第 15 页；点击查看相邻原文对照页

13 - 量化感知训练（Quantization-aware Training） 在模拟量化器 训练后量化（Post-training Quantization） 时进行训练，修改经过训练的 模型（Model）。 量化感知训练（Quantization-aware Training） (QAT) 相反，将优化过程暴露于 量化（Quantization） 效果。

典型的 QAT 前向传播（Forward Pass） 使用假 量化（Quantization）：值（Values） 是 量化后的（Quantized） 到离散代码，并立即反量化回 浮点（Floating Point），因此网络会经历 舍入（Rounding）/截断（Clipping） 行为，而 训练（Training） 机器仍然可以更新浮点（Floating-point） 参数。

从概念上讲，x→ q = Q(x)→ ˆx = D(q)→ 网络的其余部分。

回合操作通常不可微分，因此实际的 QAT 使用代理 梯度（Gradient） 行为，例如直通估计器或特定于框架的假量化模块。

QAT 可以以困难的位宽恢复 准确性（Accuracy），因为 模型（Model） 围绕量化器进行学习。

代价是它需要 训练（Training）/微调（Fine-tuning） 流水线（Pipeline）、数据和计算。因此，当质量仍然可以接受时，大型 模型（Model） 部署通常更喜欢 PTQ。

雅各布等人。将 量化（Quantization） 与 训练（Training） 结合起来，以启用仅整数算术的 推理（Inference），而当前的 PyTorch API 公开了用于 QAT 工作流程的 fake-量化（Quantization） 和观察者模块。

理解框架（Mental Model）.PTQ 询问“经过训练的 模型（Model） 在量化器中的生存能力如何？”QAT 询问“训练（Training） 可以使 模型（Model） 适应量化器吗？”研究锚点：[1]、[13] 理解 量化（Quantization）

13 - QUANTIZATION-AWARE TRAINING

Train while simulating the quantizer

Post-training quantization modifies a trained model. Quantization-aware training (QAT) instead exposes the optimization process to quantization effects.

A typical QAT forward pass uses **fake quantization**: values are quantized to discrete codes and immediately dequantized back to floating point so the network experiences rounding/clipping behavior while training machinery can still update floating-point parameters.

Conceptually,

$$x \to q = Q(x) \to \hat{x} = D(q) \to \text{rest of network.}$$

The round operation is not ordinarily differentiable, so practical QAT uses surrogate gradient behavior such as straight-through estimators or framework-specific fake-quant modules.

QAT can recover accuracy at difficult bit-widths because the model learns around the quantizer. The cost is that it requires a training/fine-tuning pipeline, data, and compute. Large-model deployment therefore often prefers PTQ when quality remains acceptable.

Jacob et al. combined quantization with training to enable integer-arithmetic-only inference, while current PyTorch APIs expose fake-quantization and observer modules for QAT workflows.

Mental model. PTQ asks "how well can this trained model survive a quantizer?" QAT asks "can training adapt the model to the quantizer?"

Research anchors: [1], [13]

14 - 仅整数与去量化执行

对应原文第 16 页：点击查看相邻原文对照页

14 - INTEGER-ONLY VS DEQUANTIZED EXECUTION 存储格式不能唯一确定算术格式 量化后的（Quantized） 张量（Tensor） 可以参与多个执行路径。

在整数 流水线（Pipeline） 中，输入和权重可以表示为整数，矩阵乘法（Matrix Multiplication） 累积为更宽的整数类型，比例因子将结果转换到下一个数值域。雅各布等人。为仅整数算术 推理（Inference） 开发了这种样式。

在许多仅包含权重的 LLM 内核（Kernels） 中，打包的 低比特（Low-bit） 权重是从内存中读取的，在 内核（Kernel） 内解包/反量化，并与更高的 精度（Precision） 激活（Activations） 相乘。即使算术路径不是“到处都是 INT4”，内存流量（Memory Traffic） 也是 低比特（Low-bit）。现代硬件还可以为 INT8、FP8 或 FP4 样式格式提供本机低 精度（Precision） 矩阵（Matrix） 引擎。哪条路径最快取决于设备支持、张量形状（Tensor Shape）、打包（Packing）、内核（Kernel） 融合以及是否隐藏转换开销。

包装后的 低比特（Low-bit） 重量 拆包/称重 矩阵乘法（Matmul）/累加 输出

重要。检查点的存储位宽、内核（Kernel） 输入位宽、乘法器格式和 累加器（Accumulator） 格式都可以不同。

研究锚点：[1]、[4]、[9]、[14]、[15] 理解 量化（Quantization）

14 - INTEGER-ONLY VS DEQUANTIZED EXECUTION

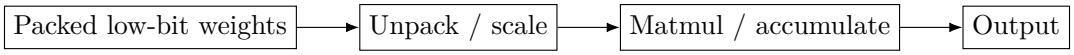
Storage format does not uniquely determine arithmetic format

A quantized tensor can participate in several execution paths.

In an integer pipeline, inputs and weights may be represented as integers, matrix multiplication accumulates into a wider integer type, and scale factors convert the result to the next numerical domain. Jacob et al. developed this style for integer-arithmetic-only inference.

In many LLM weight-only kernels, packed low-bit weights are read from memory, unpacked/dequantized inside the kernel, and multiplied with higher-precision activations. The memory traffic is low-bit even though the arithmetic path is not "INT4 everywhere."

Modern hardware can also provide native low-precision matrix engines for INT8, FP8, or FP4-style formats. Which path is fastest depends on device support, tensor shape, packing, kernel fusion, and whether conversion overhead is hidden.



Important. A checkpoint's storage bit-width, the kernel input bit-width, the multiplier format, and the accumulator format can all be different.

Research anchors: [1], [4], [9], [14], [15]

15 - 仅权重量化（Weight-only Quantization）

对应原文第 17 页；点击查看相邻原文对照页

15 - 仅权重量化（Weight-only Quantization） 压缩 模型（Model） 而不强制 激活（Activations） 为低 仅权重量化（Weight-only Quantization） 对于自回归 LLM 推理（Inference） 很有吸引力，因为 模型（Model） 权重很大并且通过 矩阵（Matrix） 乘法重复流式传输，特别是在小解码批量大小时。

诸如 W4A16 之类的符号通常意味着权重存储在 4 位 表示（Representation） 中，而 激活（Activations） 保留在 FP16/BF16 精度（Precision） 附近。 内核（Kernel） 可以加载打包的 4 位权重，在芯片上对其进行反量化/缩放，并且 合并（Combine） 可以使用更高的 精度（Precision） 激活（Activations）。

好处包括：

- 较低的检查点和设备-内存占用（Memory Footprint）；

- 词元（Token） 的权重带宽更低；

- 较大的模型（Models） 安装在固定设备上；

- 当内核（Kernel） 为带宽受限（Bandwidth-bound） 并经过优化时，潜在的延迟（Latency）/吞吐量（Throughput） 增益。

但 仅权重量化（Weight-only Quantization） 不会收缩 激活（Activation） 内存、KV 缓存内存或每个中间张量（Tensor）。它也不能保证成比例的计算加速。

GPTQ 和 AWQ 是两个著名的 低比特（Low-bit） 仅重量系列。它们的不同之处在于如何选择 量化后的（Quantized） 权重/秤，但两者都依赖专门的 内核（Kernels） 将压缩转换为部署 性能（Performance）。

理解框架（Mental Model）。 仅权重量化（Weight-only Quantization） 攻击用于表示和移动 模型（Model） 参数的字节。它不会量化整个 推理（Inference） 状态。

研究锚点：[6]、[7]、[15] 理解 量化（Quantization）

15 - WEIGHT-ONLY QUANTIZATION

Compress the model without forcing activations low

Weight-only quantization is attractive for autoregressive LLM inference because model weights are large and repeatedly streamed through matrix multiplications, especially at small decode batch sizes.

A notation such as W4A16 usually means the weights are stored in a 4-bit representation while activations remain around FP16/BF16 precision. The kernel can load packed 4-bit weights, dequantize/scaled them on chip, and combine them with higher-precision activations.

Benefits can include:

- lower checkpoint and device-memory footprint;
- lower weight bandwidth per token;
- larger models fitting on a fixed device;
- potential latency/throughput gains when the kernel is bandwidth-bound and optimized.

But weight-only quantization does not shrink activation memory, KV-cache memory, or every intermediate tensor. It also does not guarantee a proportional compute speedup.

GPTQ and AWQ are two prominent low-bit weight-only families. They differ in how they choose the quantized weights/scales, but both rely on specialized kernels to convert compression into deployment performance.

Mental model. Weight-only quantization attacks the bytes used to represent and move model parameters. It does not quantize the entire inference state.

Research anchors: [6], [7], [15]

16 - 为什么“4 位 模型（Model）”不是 4 位计算

对应原文第 18 页；点击查看相邻原文对照页

16 - 为什么“4 位 模型（Model）”不是 4 位计算 存储、元数据和 累加（Accumulation） 在口号中仍然存在假设 模型（Model） 名义上具有 4 位权重。仍然可以保留几个更高的 精度（Precision） 对象：

- 逐组（Per-group） 刻度和可能的零点；

- 偏差和归一化（Normalization） 参数；

- 嵌入向量（Embeddings） 或选定的层保持在较高的精度（Precision）；

- FP16/BF16 中的 激活（Activations）；

- 累加器为 INT32、FP16、BF16 或 32 位浮点（FP32），具体取决于 内核（Kernel）；

- 键值缓存（KV Cache） 为另一个数据类型；

- 内核（Kernel） 内的临时反量化片段。

QLoRA 是说明这种语言为何如此重要的一个清晰示例。它将冻结的基础 模型（Model） 存储在 4 位 NF4 表示（Representation） 中，但 训练（Training） 的计算使用更高的 精度（Precision） 计算数据类型，并且梯度流到 LoRA 适配器中，而不是将整个 训练（Training） 过程转换为 4 位算术。

同样，W4A16 TensorRT-LLM 配方明确命名了权重和 激活（Activations） 的不同精度。

因此，短语“4 位 模型（Model）”最好理解为特定存储/量化（Quantization） 合约的简写，而不是声称内存中的每个数字和每个乘法都是四位宽。

重要：比较 量化后的（Quantized） 系统时，要求提供完整的 精度（Precision） 元组：权重、激活（Activations）、累加器、键值缓存（KV Cache）、比例和任何未量化的异常。

研究锚点：[8]、[15]、[16] 理解 量化（Quantization）

16 - WHY “4-BIT MODEL” IS NOT 4-BIT COMPUTE

Storage, metadata, and accumulation survive the slogan

Suppose a model has nominally 4-bit weights. Several higher-precision objects can still remain:

- per-group scales and possibly zero-points;
- biases and normalization parameters;
- embeddings or selected layers kept at higher precision;
- activations in FP16/BF16;
- accumulators in INT32, FP16, BF16, or FP32 depending on the kernel;
- KV cache at another dtype;
- temporary dequantized fragments inside the kernel.

QLoRA is a clean example of why this language matters. It stores the frozen base model in a 4-bit NF4 representation, but computation for training uses a higher-precision compute dtype and gradients flow into LoRA adapters rather than turning the whole training process into 4-bit arithmetic.

Likewise, W4A16 TensorRT-LLM recipes explicitly name different precisions for weights and activations.

So the phrase “4-bit model” is best read as a shorthand for a particular storage/quantization contract, not as a claim that every number in memory and every multiply is four bits wide.

Important. When comparing quantized systems, ask for the full precision tuple: weights, activations, accumulators, KV cache, scales, and any unquantized exceptions.

Research anchors: [8], [15], [16]

17 - 激活（Activation） 异常值

对应原文第 19 页；点击查看相邻原文对照页

17 - 激活（Activation） 异常值 一些大维度可以为许多普通 值（Values） LLM 设置比例，激活（Activations） 可以包含持久的高幅度特征。如果在 张量（Tensor） 或 向量（Vector） 之间共享一个尺度，则这些异常值会扩展所需的 动态范围（Dynamic Range） 并使大多数较小的 值（Values） 的 量化（Quantization） 步长变得更粗。

这就是为什么 激活量化（Activation Quantization） 比重量 量化（Quantization） 更硬。 激活（Activation） 分布也会随着输入和运行时状态而变化，而权重在 训练（Training） 之后是固定的。

几种有影响力的方法有不同的反应：

- LLM.int8() 隔离 离群值（Outlier） 特征维度并在更高的 精度（Precision） 中计算它们，而大多数 值（Values） 使用 INT8；

- SmoothQuant 通过等效变换重新调整 激活（Activations） 和权重，使 激活（Activation） 离群值变得更容易量化；

- AWQues 激活（Activation） 统计数据来识别量化误差（Quantization Error） 最重要的权重通道，然后重新缩放以保护它们。

这些不是可以互换的算法。他们分享了一个教训：矩阵（Matrix） 乘法所看到的数值分布比单独的标称位宽更重要。

理解框架（Mental Model）。异常值的代价是昂贵的，因为共享尺度会将局部极端值变成全局分辨率损失。

研究锚点：[3]、[5]、[7] 理解 量化（Quantization）

17 - ACTIVATION OUTLIERS

A few large dimensions can set the scale for many ordinary values

LLM activations can contain persistent high-magnitude features. If one scale is shared across a tensor or vector, these outliers expand the required dynamic range and make the quantization step coarser for the majority of smaller values.

This is why activation quantization can be harder than weight quantization. The activation distribution also changes with input and runtime state, while weights are fixed after training.

Several influential methods respond differently:

- **LLM.int8()** isolates outlier feature dimensions and computes them in higher precision while most values use INT8;
- **SmoothQuant** rescales activations and weights with an equivalent transformation so activation outliers become easier to quantize;
- **AWQ** uses activation statistics to identify weight channels whose quantization error matters most, then rescales to protect them.

These are not interchangeable algorithms. They share a lesson: the numerical distribution seen by the matrix multiply matters more than nominal bit-width alone.

Mental model. Outliers are expensive because shared scales turn a local extreme into a global loss of resolution.

Research anchors: [3], [5], [7]

18 - LLM.INT8()

对应原文第 20 页；点击查看相邻原文对照页

18 - LLM.INT8() 将几乎所有 矩阵乘法（Matrix Multiplication） 保留在 INT8 中，隔离异常值 LLM.int8() 是为大型 Transformer 模型（Transformer） 推理（Inference） 设计的，其中天真的 INT8 量化（Quantization） 会随着 模型（Model） 规模的增加而降低质量。

该方法结合了两种想法。首先，大多数 值（Values） 使用 向量（Vector）-wise INT8 量化（Quantization） 以及用于内积的单独 归一化（Normalization） 常量。其次，被识别为异常值的特征维度在更高的精度（Precision）中被分离和处理。该论文报告称，超过 99.9% 的 值（Values） 仍然可以参与 8 位路径，而 离群值（Outlier） 维度则使用 16 位 矩阵乘法（Matrix Multiplication）。

这是混合 精度（Precision） 分解，而不是声称整个 模型（Model） 是纯 INT8 算术。

系统结果对于简单化的速度主张也是一个有用的警告。该论文的补充基准测试表明，量化（Quantization）/反量化（Dequantization） 开销可以主导较小的 矩阵（Matrix） 乘法，而较大的 模型（Model） 维度从 INT8 执行中受益更多。

重要： LLM.int8() 的教训比一种算法更广泛：低比特（Low-bit） 算术仅在以硬件感知方式处理数值异常和转换开销时才有帮助。

研究锚：[3]

了解 量化（Quantization）

18 - LLM.INT8()

Keep almost all matrix multiplication in INT8, isolate outliers

LLM.int8() was designed for large Transformer inference where naive INT8 quantization degraded quality as model scale increased.

The method combines two ideas. First, most values use vector-wise INT8 quantization with separate normalization constants for inner products. Second, feature dimensions identified as outliers are separated and processed in higher precision. The paper reports that more than 99.9% of values can still participate in the 8-bit path while outlier dimensions use a 16-bit matrix multiplication.

This is a mixed-precision decomposition, not a claim that the whole model is pure INT8 arithmetic.

The systems results are also a useful warning against simplistic speed claims. The paper's supplemental benchmarks show that quantization/dequantization overhead can dominate smaller matrix multiplications, while larger model dimensions benefit more from INT8 execution.

Important. The lesson from LLM.int8() is broader than one algorithm: low-bit arithmetic only helps when numerical exceptions and conversion overhead are handled in a hardware-aware way.

Research anchors: [3]

19 - 平滑量化

对应原文第 21 页；点击查看相邻原文对照页

19 - SMOOTHQUANT 将 量化（Quantization） 难度从 激活（Activations） 移至权重 SmoothQuant 目标为 W8A8 训练后量化（Post-training Quantization） 以进行 LLM 矩阵（Matrix） 乘法。它的 核心（Core） 观察结果是，权重通常比具有强异常值的 激活（Activations） 更容易量化。

对于 激活（Activation） 通道 Xj 和相应的权重通道 Wj，SmoothQuant 引入 逐通道（Per-channel） 缩放（Scaling） 因子 j 并应用以下形式的代数等效变换

完整的 精度（Precision） 产品在 量化（Quantization） 之前没有变化。然而，从数值上看，当重量幅度增加时，激活（Activation） 幅度可以减小，从而将部分 量化（Quantization） 难度从 激活（Activations） 转移到重量上。

平滑强度由特定于方法的统计数据/超参数控制。该转换是根据 校准（Calibration） 数据离线计算的，因此部署不需要重新训练原始 模型（Model）。

SmoothQuant 公布的加速与其 INT8 内核（Kernels）、模型（Models） 和硬件相关。持久概念是等效的重新缩放，而不是通用的 延迟（Latency） 乘数。

理解框架（Mental Model）。有时，最好的量化器会在 舍入（Rounding） 之前更改 表示（Representation），而不是接受原始的 张量（Tensor） 分布。

研究锚：[5]

了解 量化（Quantization）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

19 - SMOOTHQUANT

Move quantization difficulty from activations into weights

SmoothQuant targets W8A8 post-training quantization for LLM matrix multiplications. Its core observation is that weights are typically easier to quantize than activations with strong outliers.

For an activation channel X_j and corresponding weight channel W_j, SmoothQuant introduces a per-channel scaling factor s_j and applies an algebraically equivalent transformation of the form

$$XW = (X\ \mathrm{diag}(s)^{-1})\,(\mathrm{diag}(s)W).$$

The full-precision product is unchanged before quantization. Numerically, however, activation magnitudes can be reduced while weight magnitudes grow, transferring part of the quantization difficulty from activations to weights.

The smoothing strength is controlled by method-specific statistics/hyperparameters. The transform is computed offline from calibration data, so deployment does not require retraining the original model.

SmoothQuant's published speedups are tied to its INT8 kernels, models, and hardware. The durable concept is the equivalent rescaling, not a universal latency multiplier.

Mental model. Sometimes the best quantizer changes the representation before rounding rather than accepting the original tensor distribution.

Research anchors: [5]

20 - 零定量

对应原文第 22 页；点击查看相邻原文对照页

20 - ZEROQUANT 量化（Quantization） 也是一个后端问题 ZeroQuant 很有用，因为它明确了一个经常被忽略的点：量化（Quantization） 算法和高效的 量化后的（Quantized） 系统是解决方案的不同层。

该方法将细粒度权重/激活量化（Activation Quantization） 与分层知识蒸馏选项和旨在消除 量化（Quantization）/反量化（Dequantization） 开销的优化后端相结合。在论文的实验中，相对于 FP16，INT8 权重 激活（Activation） 推理（Inference） 和混合 INT4/INT8 方案实现了 模型（Model） 和硬件特定的效率改进。

确切的 基准测试（Benchmark） 数字不如解决方案的架构重要：

1.选择量化（Quantization）粒度/范围；

2. 如果需要恢复质量；

3. 实施 内核（Kernels），防止转换开销抹去节省的成本。

如果运行时在 全局内存（Global Memory） 中重复实现反量化权重、启动额外的 内核（Kernels） 或回退到不受支持的 算子（Operators），则数学上良好的 低比特（Low-bit） 表示（Representation） 仍然可能很慢。

理解框架（Mental Model）。 量化（Quantization） 有算法的一半和系统的一半。没有执行路径的压缩不是 推理（Inference） 优化。

研究锚：[4]

了解 量化（Quantization）

20 - ZEROQUANT

Quantization is also a backend problem

ZeroQuant is useful because it makes an often-missed point explicit: a quantization algorithm and an efficient quantized system are different layers of the solution.

The method combines fine-grained weight/activation quantization with layer-wise knowledge distillation options and an optimized backend intended to remove quantization/dequantization overhead. In the paper's experiments, INT8 weight-activation inference and mixed INT4/INT8 schemes achieved model- and hardware-specific efficiency improvements relative to FP16.

The exact benchmark numbers are less important than the architecture of the solution:

1. choose quantization granularity/ranges;
2. recover quality if needed;
3. implement kernels that keep conversion overhead from erasing the savings.

A mathematically good low-bit representation can still be slow if the runtime repeatedly materializes dequantized weights in global memory, launches extra kernels, or falls back to unsupported operators.

Mental model. Quantization has an algorithmic half and a systems half. Compression without an execution path is not an inference optimization.

Research anchors: [4]

21 - GPTQ

对应原文第 23 页；点击查看相邻原文对照页

21 - GPTQ 使用近似二阶信息的一次性权重 量化（Quantization） GPTQ 是一种后 训练（Training） 权重量化（Quantization） 方法，旨在将非常大的生成 Transformer 压缩到低位宽，而无需完全重新训练。

在较高层次上，它逐层量化权重，同时补偿早期 量化后的（Quantized） 权重引入的误差。更新使用推导的近似二阶信息

使该程序在法学硕士规模上切实可行。

这与简单地 舍入（Rounding） 每个权重独立不同。 GPTQ 尝试在代表性输入下最小化 量化（Quantization） 对层输出的影响。

原始论文演示了非常大的 GPT/OPT 样式 模型（Models） 的 3 位和 4 位权重 量化（Quantization），并包括自定义 低比特（Low-bit） 内核（Kernels）。现代实现添加了详细信息，例如 组大小（Group Size）、激活（Activation） 排序、静态组、打包（Packing） 布局和后端特定的 内核（Kernels）。

重要：“GPTQ 4 位”不是一种通用的二进制格式。 准确性（Accuracy） 和运行时取决于 组大小（Group Size）、对称/非对称选择、内核（Kernel） 实现和 校准（Calibration） 配置。

研究锚：[6]

了解 量化（Quantization）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

21 - GPTQ

One-shot weight quantization using approximate second-order information

GPTQ is a post-training weight quantization method designed to compress very large generative Transformers to low bit-widths without full retraining.

At a high level, it quantizes weights layer by layer while compensating for the error introduced by earlier quantized weights. The update uses approximate second-order information derived from calibration activations, following the Optimal Brain Quantization lineage but reorganized to make the procedure practical at LLM scale.

This is different from simply rounding every weight independently. GPTQ tries to minimize the effect of quantization on the layer's outputs under representative inputs.

The original paper demonstrated 3- and 4-bit weight quantization for very large GPT/OPT-style models and included custom low-bit kernels. Modern implementations add details such as group size, activation ordering, static groups, packing layout, and backend-specific kernels.

Important. "GPTQ 4-bit" is not one universal binary format. Accuracy and runtime depend on group size, symmetric/asymmetric choices, kernel implementation, and calibration configuration.

Research anchors: [6]

22 - 预警

对应原文第 24 页；点击查看相邻原文对照页

22 - AWQ 使用 激活（Activations） 来识别哪些权重通道最重要 AWQ 是另一种后 训练（Training）、低比特（Low-bit） 仅权重方法，但其机制与 GPTQ 不同。

该论文指出，量化误差（Quantization Error） 在各个权重通道中的重要性并不相同。

激活（Activation） 统计数据用于识别显着通道，等效的 逐通道（Per-channel） 缩放（Scaling） 变换可减少重要权重的误差，而无需存储混合的 精度（Precision） 受保护权重集。

AWQ 刻意避免反向传播和层重建。其 校准（Calibration） 进程从 激活（Activation） 统计数据中搜索 缩放（Scaling） 因子，然后将生成的权重打包为硬件友好的 推理（Inference）。

这给出了有用的概念对比：

- GPTQ 使用重建/二阶误差补偿程序；

- AWQ 使用 激活（Activation） 感知 缩放（Scaling） 来保护显着的权重通道；

- 两者通常都部署为低比特（Low-bit） 仅权重量化（Weight-only Quantization） 和专门的内核（Kernels）。

已发布的 TinyChat 加速特定于本文的设备和实现，而不是孤立的 AWQ 位宽属性。

理解框架（Mental Model）。 量化（Quantization） 方法决定要避免哪些错误； 内核（Kernel） 决定压缩后的 表示（Representation） 是否成为 性能（Performance） 胜利。

研究锚：[7]

了解 量化（Quantization）

22 - AWQ

Use activations to identify which weight channels matter most

AWQ is another post-training, low-bit weight-only method, but its mechanism differs from GPTQ.

The paper observes that quantization error is not equally important across weight channels. Activation statistics are used to identify salient channels, and an equivalent per-channel scaling transformation reduces the error on important weights without storing a mixed-precision set of protected weights.

AWQ deliberately avoids backpropagation and layer reconstruction. Its calibration process searches scaling factors from activation statistics, then the resulting weights are packed for hardware-friendly inference.

This gives a useful conceptual contrast:

- GPTQ uses a reconstruction/second-order error-compensation procedure;
- AWQ uses activation-aware scaling to protect salient weight channels;
- both are commonly deployed as low-bit weight-only quantization with specialized kernels.

Published TinyChat speedups are specific to the paper's devices and implementation, not a property of AWQ bit-width in isolation.

Mental model. The quantization method decides which errors to avoid; the kernel decides whether the compressed representation becomes a performance win.

Research anchors: [7]

23 - GPTQ、AWQ、SMOOTHQUANT、LLM.INT8

对应原文第 25 页；点击查看相邻原文对照页

23 - GPTQ、AWQ、SMOOTHQUANT、LLM.INT8 类似的 词表（Vocabulary），不同的问题 方法 主要目标 核心（Core） 想法 典型部署框架 LLM.int8() W/A 矩阵乘法（Matmul） 隔离 离群值（Outlier） 维度混合 INT8/16 位 SmoothQuant W8A8 迁移 激活（Activation） 难度到权重 INT8 矩阵乘法（Matmul） GPTQ 使用二阶信息进行权重重建 W3/W4A16 式 AWQ 权重 激活（Activation） 感知通道 缩放（Scaling） W4A16 式 该表是有意设计的。真正的框架支持多种变体和位宽。

三个区别可以防止混淆：

1. 仅权重与权重-激活（Activation）：减少 模型（Model） 带宽与启用 低比特（Low-bit） 激活（Activation） matmuls 不同；

2.算法 vs 格式：GPTQ 和 AWQ 是方法，而不是原始数值数据类型；

3. 量化器与运行时：相同的 量化后的（Quantized） 检查点在 内核（Kernels） 和硬件上的执行情况可能不同。

重要：不要问“GPTQ 比 INT4 更好吗？”GPTQ是一种产生低比特（Low-bit）权重的方法； INT4 描述了一个数字/存储系列。这种比较混合了抽象级别。

研究锚点：[3]、[5]-[7] 理解 量化（Quantization）

23 - GPTQ, AWQ, SMOOTHQUANT, LLM.INT8

Similar vocabulary, different problems

Method	Main target	Core idea	Typical deployment framing
LLM.int8()	W/A matmul	isolate outlier dimensions	mixed INT8/16-bit
SmoothQuant	W8A8	migrate activation difficulty to weights	INT8 matmul
GPTQ	weights	reconstruction with second-order information	W3/W4A16-style
AWQ	weights	activation-aware channel scaling	W4A16-style

The table is intentionally conceptual. Real frameworks support multiple variants and bit-widths.

Three distinctions prevent confusion:

1. **weight-only vs weight-activation:** reducing model bandwidth is different from enabling low-bit activation matmuls;
2. **algorithm vs format:** GPTQ and AWQ are methods, not primitive numerical data types;
3. **quantizer vs runtime:** the same quantized checkpoint can perform differently across kernels and hardware.

Important. Do not ask "is GPTQ better than INT4?" GPTQ is a way to produce low-bit weights; INT4 describes a numerical/storage family. The comparison mixes levels of abstraction.

Research anchors: [3], [5]-[7]

24 - NF4

对应原文第 26 页；点击查看相邻原文对照页

用于正态分布权重的 4 位 码本（Codebook） QLoRA 引入了 NormalFloat 4 (NF4)，用于量化预训练权重，这些权重在 归一化（Normalization） 之后按局部绝对最大值近似正态分布。

NF4 使用从正态分布的分位数中选择的 16 个可表示的 码本（Codebook） 值（Values） 并标准化为有界范围。因此，存储的 4 位 值（Value） 是该 码本（Codebook） 中的 索引（Index）。

它不是带有符号、指数和尾数字段的传统 浮点（Floating-point） 编码。

当前的 bitsandbytes 源明确说明了这一点：NF4 和 FP4 使用不同的码本/编码，即使每个存储的 索引（Index） 都消耗 4 位。

非均匀 码本（Codebook） 级别（原理图）

理解框架（Mental Model）。 NF4 通过选择根据权重分布假设定制的非均匀 16 重建组 值（Values） 进行压缩。

了解 量化（Quantization） 研究锚点：[8]、[16]、[17] QLoRA 将 NF4 描述为在其规定的结构下对于正态分布权重而言信息理论上最优的。后来的技术说明认为，严格的最优性并不适用于有限块absmax 归一化（Normalization），因为后归一化（Normalization） 分布取决于块大小。将原始主张视为特定于方法的动机，而不是通用属性。 [8,17]

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

24 - NF4

A 4-bit codebook for normally distributed weights

QLoRA introduced NormalFloat 4 (NF4) for quantizing pretrained weights that are approximately normally distributed after normalization by a local absolute maximum.

NF4 uses 16 representable codebook values chosen from quantiles of a normal distribution and normalized into a bounded range. A stored 4-bit value is therefore an index into this codebook. It is not a conventional floating-point encoding with sign, exponent, and mantissa fields.

Current bitsandbytes source makes this explicit: NF4 and FP4 use different codebooks/encodings, even though both consume four bits per stored index.

QLoRA describes NF4 as information-theoretically optimal for normally distributed weights under its stated construction. A later technical note argues that strict optimality does not carry through finite-block absmax normalization because the post-normalization distribution depends on block size. Treat the original claim as method-specific motivation, not a universal property. [8,17]

non-uniform codebook levels (schematic)

Mental model. NF4 compresses by choosing a non-uniform set of 16 reconstruction values tailored to a weight-distribution assumption.

Research anchors: [8], [16], [17]

25 - QLORA 和双 量化（Quantization）

对应原文第 27 页；点击查看相邻原文对照页

25 - QLORA 和双 量化（Quantization） 量化冷冻基础 模型（Model），训练小型适配器 QLoRA 将 低比特（Low-bit） 存储与参数高效的 微调（Fine-tuning） 相结合。预训练的基础 模型（Model） 被冻结并存储在 4 位 NF4 中，而梯度则通过其反量化计算反向传播到可训练的 LoRA 适配器中。

另外两个想法很重要：

双量化对 量化（Quantization） 常量本身进行量化，从而减少比例元数据开销。

分页优化器使用统一内存分页来管理 微调（Fine-tuning） 期间的瞬时优化器内存峰值。

计算数据类型仍然高于 4 位存储的 精度（Precision）。这就是为什么在没有限定的情况下将 QLoRA 描述为“训练（Training） in 4-bit”会产生误导。

QLoRA 的贡献也是一个有用的提醒，量化（Quantization） 可用于使 训练（Training）/微调内存变得可行，而不仅仅是 推理（Inference） 部署。

重要： NF4属于冷冻基本重量的存储/量化（Quantization） 表示（Representation）。

LoRA 适配器参数、优化器状态、梯度、激活（Activations） 和计算数据类型遵循自己的 精度（Precision） 规则。

研究锚点：[8]、[16] 理解 量化（Quantization）

25 - QLORA AND DOUBLE QUANTIZATION

Quantize the frozen base model, train small adapters

QLoRA combines low-bit storage with parameter-efficient fine-tuning. The pretrained base model is frozen and stored in 4-bit NF4, while gradients are backpropagated through its dequantized computations into trainable LoRA adapters.

Two additional ideas matter:

Double quantization quantizes the quantization constants themselves, reducing scale metadata overhead.

Paged optimizers use unified-memory paging to manage transient optimizer-memory spikes during fine-tuning.

The compute dtype remains higher precision than the 4-bit storage. This is why describing QLoRA as "training in 4-bit" without qualification is misleading.

QLoRA's contribution is also a useful reminder that quantization can be used to make *training/fine-tuning memory* feasible, not only inference deployment.

Important. NF4 belongs to the storage/quantization representation of the frozen base weights. LoRA adapter parameters, optimizer states, gradients, activations, and compute dtypes follow their own precision rules.

Research anchors: [8], [16]

26 - FP8：E4M3 和 E5M2

对应原文第 28 页；点击查看相邻原文对照页

26 - FP8：E4M3 和 E5M2 浮点（Floating-point） 量化（Quantization） 将 精度（Precision） 换成 动态范围（Dynamic Range） 广泛使用的 FP8 提案定义了两种 8 位 浮点（Floating-point） 格式：

- E4M3：1 个符号位、4 个指数位、3 个尾数位；

- E5M2：1 个符号位、5 个指数位、2 个尾数位。

E5M2 具有更宽的 动态范围（Dynamic Range） 和更低有效位的 精度（Precision）。 E4M3在较窄的范围内有更多的精度（Precision）。

与一标度下的仿射 INT8 不同，FP8 值（Values） 在实数空间中不是均匀分布的。

指数随幅度改变步长。

最初的 FP8 工作展示了跨神经架构的 训练（Training） 和 推理（Inference），包括大语言 模型（Models），并使用围绕这些格式的 缩放（Scaling） 策略。 NVIDIA Transformer 引擎（Transformer Engine）在支持的 GPU 上使用 E4M3/E5M2 配方，通常将敏感操作保持在较高的 精度（Precision） 下。

这就是为什么“FP8 模型（Model）”不应被理解为“每个操作都在 FP8 中执行”。 层归一化（Layer Normalization）、Softmax 归一化（Softmax）、减少、累积和不支持的操作可以保留为 BF16/FP16/32 位浮点（FP32），具体取决于配方。

理解框架（Mental Model）。整数量化（Quantization）使用缩放均匀网格； FP8 使用 浮点（Floating-point） 网格，其间距随指数变化。

研究锚点：[9]、[14] 理解 量化（Quantization）

26 - FP8: E4M3 AND E5M2

Floating-point quantization trades precision for dynamic range

The widely used FP8 proposal defines two 8-bit floating-point formats:

- **E4M3**: 1 sign bit, 4 exponent bits, 3 mantissa bits;
- **E5M2**: 1 sign bit, 5 exponent bits, 2 mantissa bits.

E5M2 has wider dynamic range and less significand precision. E4M3 has more precision within a narrower range.

Unlike affine INT8 under one scale, FP8 values are not uniformly spaced in real-number space. The exponent changes the step size with magnitude.

The original FP8 work demonstrated training and inference across neural architectures, including large language models, using scaling strategies around these formats. NVIDIA Transformer Engine uses E4M3/E5M2 recipes on supported GPUs and commonly keeps sensitive operations at higher precision.

This is why "FP8 model" should not be read as "every operation executes in FP8." Layer normalization, softmax, reductions, accumulations, and unsupported operations can remain BF16/FP16/FP32 depending on the recipe.

Mental model. Integer quantization uses a scaled uniform grid; FP8 uses a floating-point grid whose spacing changes with exponent.

Research anchors: [9], [14]

27 - FP8 缩放（Scaling） 和 混合精度（Mixed Precision）

对应原文第 29 页；点击查看相邻原文对照页

27 - FP8 缩放（Scaling） 和 混合精度（Mixed Precision） 该格式只是 FP8 配方的一半 FP8 张量（Tensor） 限制了 动态范围（Dynamic Range），因此实际系统会维护将更高 精度（Precision） 张量（Tensor） 映射到可用 FP8 范围的比例信息。

Transformer 引擎（Transformer Engine）公开了多种策略，包括延迟的 缩放（Scaling）、当前的 逐张量（Per-tensor） 缩放（Scaling） 和块式变体。典型的尺度源自观察到的绝对最大值 (amax) 和最大可表示 FP8 值（Value），其边际/历史取决于配方。

混合精度（Mixed Precision） 仍然至关重要。当前的 Transformer 引擎（Transformer Engine）文档指出，线性操作可以在低 精度（Precision） 下运行，而 归一化（Normalization）、Softmax 归一化（Softmax） 和其他操作默认情况下仍保持在较高的 精度（Precision） 下； 注意力机制（Attention） 精度（Precision） 还取决于配方/后端配置。

这种分离解释了为什么低 精度（Precision） 训练（Training） 可以保持稳定性：精度（Precision） 在各个操作中的花费不同，而不是强制整个图采用一种格式。

重要：诸如 E4M3 之类的数据类型名称并不完全指定 FP8 训练（Training）/推理（Inference） 系统。 缩放（Scaling） 粒度、amax 策略、累加（Accumulation） 精度（Precision） 和 算子（Operator） 覆盖范围是配方的一部分。

研究锚点：[9]、[14] 理解 量化（Quantization）

27 - FP8 SCALING AND MIXED PRECISION

The format is only half of an FP8 recipe

An FP8 tensor has limited dynamic range, so practical systems maintain scale information that maps a higher-precision tensor into the available FP8 range.

Transformer Engine exposes several strategies, including delayed scaling, current per-tensor scaling, and blockwise variants. A typical scale derives from an observed absolute maximum ($amax$) and the maximum representable FP8 value, with margins/history depending on the recipe.

Mixed precision remains essential. Current Transformer Engine documentation states that linear operations can run in low precision while normalization, softmax, and other operations remain at higher precision by default; attention precision also depends on recipe/backend configuration.

This separation explains why low-precision training can preserve stability: precision is spent differently across operations rather than forcing the entire graph into one format.

Important. A datatype name such as E4M3 does not fully specify an FP8 training/inference system. Scaling granularity, amax policy, accumulation precision, and operator coverage are part of the recipe.

Research anchors: [9], [14]

28 - 块级 FP8 和 FP4

对应原文第 30 页；点击查看相邻原文对照页

28 - 块级 FP8 和 FP4 现代硬件使规模更接近数据 新一代加速器支持更细粒度的块 缩放（Scaling）。例如，当前的 NVIDIA Transformer 引擎（Transformer Engine）文档描述了 Blackwell 级硬件上的 MXFP8，使用本地 缩放（Scaling） 表示 32 个 值（Values） 块，并将 NVFP4 作为受支持的 4 位低 精度（Precision） 配方。

原因很直观。整个 张量（Tensor） 的一个秤必须容纳其全局 动态范围（Dynamic Range）。为每个小块提供自己的比例可以减少每个 码本（Codebook） 必须覆盖的范围，这可以改进局部 精度（Precision）。

这带有额外的元数据和布局规则。块格式还可能需要不同的 量化后的（Quantized） 视图以进行行方向和列方向使用，因为在定向块分组下量化 张量（Tensor） 和量化其转置在数值上并不相同。

这些格式是特定于硬件和库的，因此它们应该具有版本范围。常青的想法是：

更细的尺度粒度⇒更好的局部拟合+更多的元数据/布局约束。

重要： NVFP4、NF4 和通用 INT4 是三种不同的数值方案。共享“4 位”并不意味着它们可以互换。

研究锚点：[14]、[15] 了解 量化（Quantization） 当前实施说明（Transformer 引擎（Transformer Engine） 2.19，2026 年 9 月）：重新检查当前文档。

28 - BLOCK-SCALED FP8 AND FP4

Modern hardware moves scales closer to the data

Current implementation note (Transformer Engine 2.19, September 2026): recheck current docs.

Newer accelerator generations support finer-grained block scaling. Current NVIDIA Transformer Engine documentation, for example, describes MXFP8 on Blackwell-class hardware using local scaling for blocks of 32 values, and NVFP4 as a supported 4-bit low-precision recipe.

The reason is intuitive. One scale for an entire tensor must accommodate its global dynamic range. Giving each small block its own scale reduces the range each codebook has to cover, which can improve local precision.

This comes with additional metadata and layout rules. Blockwise formats can also require different quantized views for row-wise and column-wise use because quantizing a tensor and quantizing its transpose are not numerically identical under directional block grouping.

These formats are hardware- and library-specific enough that they should be version-scoped. The evergreen idea is:

$$\text{finer scale granularity} \Rightarrow \text{better local fit} + \text{more metadata/layout constraints.}$$

Important. NVFP4, NF4, and generic INT4 are three different numerical schemes. Sharing "4-bit" does not make them interchangeable.

Research anchors: [14], [15]

29 - KV 缓存 量化（Quantization）

对应原文第 31 页；点击查看相邻原文对照页

29 - KV-CACHE 量化（Quantization） 压缩每个请求状态，而不是 模型（Model） 权重 在自回归解码期间，键值缓存（KV Cache） 可能会消耗 设备内存（Device Memory） 的很大一部分，以实现长上下文和高并发。量化 K/V 状态可减少每个缓存 词元（Token） 的字节数，还可以减少 注意力机制（Attention） 读取缓存时的内存带宽压力。

对于传统的 缓存有效载荷（Cache Payload），

因此减少每个元素的存储字节直接减少了理想化的 张量（Tensor） 有效负载。

但 KV 缓存 量化（Quantization） 在数值上与权重 量化（Quantization） 不同。 键（Keys） 和 值（Values） 是根据每个请求动态生成的。它们的分布可能因层、头、词元（Token） 和 模型（Model） 架构而异。 量化误差（Quantization Error） 还为每个后续解码步骤提供未来的 注意力机制（Attention） 计算。

因此，现代运行时公开了单独的 KV 缓存 量化（Quantization） 模式，包括受支持硬件上的 FP8 和更新的 低 精度（Precision） 格式。

理解框架（Mental Model）。权重 量化（Quantization） 更改共享 模型（Model） 资产。 KV 缓存 量化（Quantization） 更改不断增长的、特定于请求的 推理（Inference） 状态。

研究锚点：[10]、[11]、[15] 了解 量化（Quantization），其中 D_h 是每头 键（Key）/值（Value） 维度。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

29 - KV-CACHE QUANTIZATION

Compress per-request state, not model weights

During autoregressive decoding, the KV cache can consume a large fraction of device memory for long contexts and high concurrency. Quantizing K/V state reduces bytes per cached token and can also reduce memory-bandwidth pressure when attention reads the cache.

For a conventional cache payload,

$$M_{KV} = 2LTH_{KV}D_h b,$$

where D_h is the per-head key/value dimension.

so reducing the storage bytes per element b directly reduces the idealized tensor payload.

But KV-cache quantization is numerically different from weight quantization. Keys and values are generated dynamically from each request. Their distributions can vary by layer, head, token, and model architecture. Quantization error also feeds future attention calculations for every subsequent decode step.

Modern runtimes therefore expose separate KV-cache quantization modes, including FP8 and newer low-precision formats on supported hardware.

Mental model. Weight quantization changes a shared model asset. KV-cache quantization changes growing, request-specific inference state.

Research anchors: [10], [11], [15]

30 - 基维

对应原文第 32 页；点击查看相邻原文对照页

30 - KIVI 键（Keys） 和 值（Values） 不需要使用相同的粒度 KIVI 研究了缓存的 键（Keys） 和 值（Values） 的分布，发现一种不对称性对于非常 低比特（Low-bit） 量化（Quantization） 非常有用。

该论文报告称，键（Keys） 表现出通道式 离群值（Outlier） 结构，激励每通道 键（Key） 量化（Quantization），而 值（Values） 可以更好地处理每 词元（Token）。 KIVI 将这些选择结合在免调整的非对称 2 位 KV 缓存方法中，并在实际实现中为最近的 词元（Tokens） 在较高的 精度（Precision） 中保留了一个小的残留区域。

这是一个重要的概念课程：即使在 注意力机制（Attention） 方程中成对出现的张量也可以具有不同的数值统计量，因此也可以具有不同的最佳 量化（Quantization） 轴。

KIVI 报告的内存和 吞吐量（Throughput） 改进与其 模型（Models）、工作负载、内核（Kernel） 实现和实验设置相关。本手册的持久结果是分布驱动的粒度选择。

重要：“将 K 和 V 量化为 2 位”并不是 KIVI 的完整描述。每个通道/每个 词元（Token） 的不对称性是该方法的核心。

研究主播：[10]

了解 量化（Quantization）

30 - KIVI

Keys and values need not use the same granularity

KIVI studied the distributions of cached keys and values and found an asymmetry useful for very low-bit quantization.

The paper reports that keys exhibit channel-wise outlier structure, motivating **per-channel key quantization**, while values are better handled **per-token**. KIVI combines these choices in a tuning-free asymmetric 2-bit KV-cache method and retains a small residual region in higher precision for recent tokens in its practical implementation.

This is an important conceptual lesson: even tensors that appear as a pair in the attention equation can have different numerical statistics and therefore different best quantization axes.

KIVI's reported memory and throughput improvements are tied to its models, workloads, kernel implementation, and experimental setup. The durable result for this handbook is the distribution-driven granularity choice.

Important. "Quantize K and V to 2-bit" is not a complete description of KIVI. The per-channel/per-token asymmetry is central to the method.

Research anchors: [10]

31 - 科维量子

对应原文第 33 页；点击查看相邻原文对照页

31 - KVQUANT 长上下文缓存压缩使用的不仅仅是位宽 KVQuant 是针对长上下文 推理（Inference） 的极端 低比特（Low-bit） KV 缓存 量化（Quantization） 的另一项工作。它研究 量化（Quantization） 感知选择，例如 逐通道（Per-channel） 键（Key） 处理、非均匀 量化（Quantization）、离群值（Outlier） 处理和优化的 内核（Kernels）。

更广泛的一点是，精度（Precision） 非常低的 KV 缓存压缩成为系统协同设计问题：

- 选择保持注意力机制（Attention） 质量的数字表示（Representation）；

- 决定每个通道/词元（Token）/组存储哪些统计数据/尺度；

- 如果需要，处理异常值或最近的词元（Tokens）；

- 实施注意力机制（Attention） 内核（Kernels），它消耗压缩的K/V，而不需要昂贵的物化（Materialization）。

如果解压缩成为瓶颈或者 注意力机制（Attention） 内核（Kernel） 无法有效地使用该格式，则一种方法可以报告理论上缓存的显着减少，但仍然无法改进 推理服务（Serving） 吞吐量（Throughput）。

KVQuant 发布的“1000 万上下文”框架属于其研究设置，不应被解读为任意 模型（Models） 或硬件的通用上下文保证。

理解框架（Mental Model）。对于键值缓存（KV Cache），压缩比、注意力机制（Attention）质量和解码带宽必须一起评估。

研究锚：[11]

了解 量化（Quantization）

31 - KVQUANT

Long-context cache compression uses more than bit-width

KVQuant is another line of work on extremely low-bit KV-cache quantization for long-context inference. It studies quantization-aware choices such as per-channel key handling, non-uniform quantization, outlier treatment, and optimized kernels.

The broader point is that KV-cache compression at very low precision becomes a systems co-design problem:

- choose a numerical representation that preserves attention quality;
- decide what statistics/scales are stored per channel/token/group;
- handle outliers or recent tokens if needed;
- implement attention kernels that consume compressed K/V without expensive materialization.

A method can report a dramatic theoretical cache reduction and still fail to improve serving throughput if decompression becomes the bottleneck or the attention kernel cannot use the format efficiently.

KVQuant's published "10 million context" framing belongs to its research setup and should not be read as a universal context guarantee for arbitrary models or hardware.

Mental model. For KV cache, compression ratio, attention quality, and decode bandwidth must be evaluated together.

Research anchors: [11]

32 - 内存核算

对应原文第 34 页；点击查看相邻原文对照页

32 - 内存核算 理想的位宽节省是完整运行时占用空间的下限 对于每个存储为 b 位的 P 参数，理想的原始有效负载为 Mweights = P b 8 字节。

对于示例性的 70 亿参数 模型（Model）：

权重有效负载 十进制 GB GiB 16 位 14.0 13.04 8 位 7.0 6.52 4 位 3.5 3.26 实际 量化后的（Quantized） 检查点/运行时内存更大，因为它们可以包含比例、零点、对齐/填充、打包（Packing） 元数据、未量化层、偏差/规范、 码本（Codebook） 状态和临时缓冲区。

运行时内存还包括 激活（Activations）、键值缓存（KV Cache）、分配器开销（Allocator Overhead）、CUDA 图形/工作区，以及加载/转换期间可能的多个副本。

重要： 4 位权重有效负载大约是 16 位权重有效负载的四分之一。

这并不意味着 GPU 内存总使用量是四分之一。

研究锚点：[2]、[6]-[8]、[15]、[16] 理解 量化（Quantization）

32 - MEMORY ACCOUNTING

Ideal bit-width savings are a lower bound on the full runtime footprint

For P parameters stored at b bits each, the ideal raw payload is

$$M_{\text{weights}} = \frac{Pb}{8} \text{ bytes.}$$

For an illustrative 7-billion-parameter model:

Weight payload	Decimal GB	GiB
16-bit	14.0	13.04
8-bit	7.0	6.52
4-bit	3.5	3.26

Real quantized checkpoints/runtime memory are larger because they can include scales, zero-points, alignment/padding, packing metadata, unquantized layers, biases/norms, codebook state, and temporary buffers.

Runtime memory also includes activations, KV cache, allocator overhead, CUDA graphs/-workspaces, and possibly multiple copies during loading/conversion.

Important. A 4-bit weight payload is approximately one quarter of a 16-bit weight payload. That does not imply total GPU memory use is one quarter.

Research anchors: [2], [6]-[8], [15], [16]

33 - 工作记忆示例：组元数据

对应原文第 35 页；点击查看相邻原文对照页

33 - 工作存储器示例：组元数据 一个“4 位”权重的成本可能超过 4 有效位数（Effective Bits） 假设对称分组 4 位权重，每 128 个权重有一个 FP16 标度。

各组店：

加上 16 个刻度位。

所以有效成本是

128 = 4.125 比特/权重。

对于 70 亿个重量，理想的有效负载变为

如果该方案还为每组存储一个 FP16 零点（Zero-point），则成本将变为

在对齐或其他元数据之前，128 = 4.25 位/权重。

这仍然远低于 16 位存储，但它说明了为什么“参数计数乘以半个字节”只是一阶估计。

理解框架（Mental Model）.组大小（Group Size） 控制数字局部性和元数据摊销。较小的团体通常会提高适合度，同时增加规模/零点（Zero-point） 开销。

研究锚点：[6]-[8]、[16]理解量化（Quantization）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

33 - WORKED MEMORY EXAMPLE: GROUP METADATA

A “4-bit” weight can cost more than 4 effective bits

Assume symmetric groupwise 4-bit weights with one FP16 scale for every 128 weights.

Each group stores:

$$128 \times 4 = 512 \text{ weight bits}$$

plus

$$16 \text{ scale bits.}$$

So the effective cost is

$$4 + \frac{16}{128} = 4.125 \text{ bits/weight.}$$

For 7 billion weights, that idealized payload becomes

$$7 \times 10^9 \times \frac{4.125}{8} = 3.609375 \times 10^9 \text{ bytes} \approx 3.36 \text{ GiB.}$$

If the scheme also stored one FP16 zero-point per group, the cost would become

$$4 + \frac{32}{128} = 4.25 \text{ bits/weight}$$

before alignment or other metadata.

This is still far below 16-bit storage, but it shows why “parameter count times half a byte” is only the first-order estimate.

Mental model. Group size controls both numerical locality and metadata amortization. Smaller groups usually improve fit while increasing scale/zero-point overhead.

Research anchors: [6]-[8], [16]

34 - 内核（Kernels） 和硬件

对应原文第 36 页；点击查看相邻原文对照页

34 - 内核（Kernels） 和硬件 压缩只有在机器可以有效地消耗它时才会变得速度低-精度（Precision） 性能（Performance） 取决于完整的执行路径。

一个有用的清单是：

- GPU/加速器是否具有对该格式的本机张量（Tensor）-核心（Core）/指令支持？

- 重量是否符合内核（Kernel） 预期的布局？

- 秤是否有效加载并与反量化（Dequantization） 融合？

- 当前批次/阶段的操作是否类似于通用矩阵乘法（GEMM） 或GEMV？

- 对于不支持的形状/算子（Operators），运行时是否会回退到更高的精度（Precision） 实现？

- 转换是在芯片上执行的，还是反量化的张量写回全局内存（Global Memory）？

当前的 TensorRT-LLM 公开了不同的配方，例如 FP8、FP4、W4A16 GPTQ/AWQ、W4A8 变体和 量化后的（Quantized） KV 缓存模式。 Transformer 引擎（Transformer Engine）同样通过硬件限制来区分 FP8、MXFP8 和 NVFP4 配方。

这就是为什么 量化后的（Quantized） 检查点在错误的设备或软件堆栈上可能更小但更慢。

重要：位宽是 表示（Representation） 的属性。速度是 表示（Representation） + 算子（Operator） 形状 + 内核（Kernels） + 硬件 + 工作负载的属性。

研究锚点：[14]、[15]理解量化（Quantization）

34 - KERNELS AND HARDWARE

Compression only becomes speed when the machine can consume it efficiently

Low-precision performance depends on the complete execution path.

A useful checklist is:

- Does the GPU/accelerator have native tensor-core/instruction support for the format?
- Are weights packed in the layout expected by the kernel?
- Are scales loaded efficiently and fused with dequantization?
- Is the operation GEMM-like or GEMV-like at the current batch/phase?
- Does the runtime fall back to a higher-precision implementation for unsupported shapes/operators?
- Is conversion performed on chip, or are dequantized tensors written back to global memory?

Current TensorRT-LLM exposes distinct recipes such as FP8, FP4, W4A16 GPTQ/AWQ, W4A8 variants, and quantized KV-cache modes. Transformer Engine similarly distinguishes FP8, MXFP8, and NVFP4 recipes with hardware constraints.

This is why a quantized checkpoint can be smaller yet slower on the wrong device or software stack.

Important. Bit-width is a property of representation. Speed is a property of representation + operator shapes + kernels + hardware + workload.

Research anchors: [14], [15]

35 - 延迟（Latency）、吞吐量（Throughput） 和带宽

对应原文第 37 页：点击查看相邻原文对照页

35 - 延迟（Latency）、吞吐量（Throughput） 和带宽 较低的 精度（Precision） 以不同的方式改变不同的瓶颈 量化（Quantization） 可以帮助多个资源，但并不总是同时帮助。

容量。较小的权重/缓存允许容纳更大的 模型（Models） 或更多并发请求。

带宽。对于权重绑定解码或 KV 绑定 注意力机制（Attention） 必须加载更少的字节。

计算 吞吐量（Throughput）。本机低 精度（Precision） 张量核心（Tensor Cores） 每秒可以在支持的形状上执行更多操作。

延迟（Latency）。仅当减少的瓶颈超过 打包（Packing）/反量化（Dequantization）/启动开销（Launch Overhead） 时，端到端 延迟（Latency） 才会得到改善。

吞吐量（Throughput）。即使单个请求 延迟（Latency） 几乎没有变化，内存节省也可以提高批处理/并发性。

LLM.int8() 提供了一个具体的历史示例：较小的 矩阵（Matrix） 乘法可能会损失 量化（Quantization） 开销，而较大的维度则受益更多。 KIVI 类似地展示了缓存压缩如何增加可行的 批大小（Batch Size），这会改变 吞吐量（Throughput），即使算术图在其他方面相似。

理解框架（Mental Model）。询问移动了哪个资源：字节 驻留（Resident）、传输的字节、算术 吞吐量（Throughput）、批量容量或端到端时间。没有衡量标准的“更快”是不完整的。

研究锚点：[3]、[10]、[14]、[15] 理解 量化（Quantization）

35 - LATENCY, THROUGHPUT, AND BANDWIDTH

Lower precision changes different bottlenecks differently

Quantization can help several resources, but not always at the same time.

Capacity. Smaller weights/cache allow larger models or more concurrent requests to fit.

Bandwidth. Fewer bytes must be loaded for weight-bound decode or KV-bound attention.

Compute throughput. Native low-precision tensor cores can execute more operations per second on supported shapes.

Latency. End-to-end latency improves only if the reduced bottleneck dominates more than packing/dequantization/launch overhead.

Throughput. Memory savings can raise batch/concurrency even when single-request latency barely changes.

LLM.int8() provides a concrete historical example: smaller matrix multiplications could lose to quantization overhead, while larger dimensions benefited more. KIVI similarly shows how cache compression can increase feasible batch size, which changes throughput even if the arithmetic graph is otherwise similar.

Mental model. Ask which resource moved: bytes resident, bytes transferred, arithmetic throughput, batch capacity, or end-to-end time. “Faster” without a metric is incomplete.

Research anchors: [3], [10], [14], [15]

36 - 透明的 Pytorch 量化器

对应原文第 38 页；点击查看相邻原文对照页

36 - 透明的 PYTORCH 量化器 教学对称量化-反量化 以下代码反映了已工作的 INT8 示例。它是故意简单的，并不是优化的 推理（Inference） 内核（Kernel）。

qmax = 2 ** (位 - 1) - 1 比例 = x。 abs().max() / qmax 比例 = torch.clamp(scale, min=torch.finfo(x.dtype).eps) q = torch.clamp(

).to(torch.int8) x_hat = q.浮动（）*规模

x = torch.张量（Tensor）([-1.2, -0.5, 0.0, 0.3, 1.0]) q, 比例, x_hat = symmetry_qdq(x)

代码注释：q = [-127, -53, 0, 32, 106]

代码注释：x_hat = [-1.2000, -0.5008, 0.0000, 0.3024, 1.0016]

生产实现还必须定义粒度、dtype/范围约定、零点、打包（Packing） 布局、缩放 dtype、组大小（Group Size）、截断（Clipping）/校准（Calibration） 策略、算子（Operator） 支持和 累加（Accumulation） 精度（Precision）。

重要：该函数演示了数字地图。它不会仅仅因为 张量（Tensor） 代码是整数而使 PyTorch 使用优化的 INT8 内核（Kernel） 执行 矩阵乘法（Matrix Multiplication）。

研究锚点：[1]、[13] 理解 量化（Quantization） 对于位 < 8，q 在这里仍然使用 torch.int8；此示例未实现子字节 打包（Packing）。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

36 - A TRANSPARENT PYTORCH QUANTIZER

Pedagogical symmetric quantize-dequantize

The following code mirrors the worked INT8 example. It is intentionally simple and is not an optimized inference kernel.

```
import torch

def symmetric_qdq(x, bits=8):
    qmax = 2 ** (bits - 1) - 1
    scale = x.abs().max() / qmax
    scale = torch.clamp(scale, min=torch.finfo(x.dtype).eps)
    q = torch.clamp(
        torch.round(x / scale), -qmax, qmax
    ).to(torch.int8)
    x_hat = q.float() * scale
    return q, scale, x_hat

x = torch.tensor([-1.2, -0.5, 0.0, 0.3, 1.0])
q, scale, x_hat = symmetric_qdq(x)
# q = [-127, -53, 0, 32, 106]
# x_hat = [-1.2000, -0.5008, 0.0000, 0.3024, 1.0016]
```

For bits < 8, q still uses torch.int8 here; this example does not implement sub-byte packing.

A production implementation must additionally define granularity, dtype/range conventions, zero-points, packing layout, scale dtype, group size, clipping/calibration policy, operator support, and accumulation precision.

Important. This function demonstrates the numerical map. It does not make PyTorch execute the matrix multiplication with an optimized INT8 kernel merely because the tensor codes are integers.

Research anchors: [1], [13]

37 - 常见的实施错误

对应原文第 39 页；点击查看相邻原文对照页

37 - 常见实施错误 大多数 低比特（Low-bit） 错误是合同不匹配

1. 比较无格式的位宽。NF4、FP4、INT4 和 NVFP4 不可互换。

2. 从内存估计中删除比例元数据。分组方案需要比例，有时还需要零点。

3. 使用错误的轴。逐通道（Per-channel） 沿着错误的尺寸可能会改变 准确性（Accuracy） 和 内核（Kernel） 的兼容性。

4. 重复使用错误的 校准（Calibration） 数据。激活（Activation） 范围可能取决于工作负载。

5. 假设存储 精度（Precision） 等于计算 精度（Precision）。仅权重 内核（Kernels） 通常以更高的 精度（Precision） 进行计算/累加。

6. 忽略未量化的异常。Norms、嵌入向量（Embeddings）、输出头（Output Heads） 或选定的层可能保持高 精度（Precision）。

7. 将检查点大小作为 延迟（Latency） 进行基准测试。较小的文件并不能保证更快的 生成（Generation）。

8. 忘记打包（Packing）/布局。数学上正确的量化器可能无法被目标 内核（Kernel） 使用。

9. 混合不兼容的尺度/检查点。量化（Quantization） 元数据是 模型（Model） 合约的一部分。

10. 出版论文加速通用。硬件、批次、序列长度和软件版本必须匹配。

理解框架（Mental Model）。将 量化后的（Quantized） 检查点视为权重 + 量化（Quantization） 元数据 + 所需的执行合约。

研究锚点：[3]-[16]理解量化（Quantization）

37 - COMMON IMPLEMENTATION MISTAKES

Most low-bit bugs are contract mismatches

1. **Comparing bit-widths without format.** NF4, FP4, INT4, and NVFP4 are not interchangeable.
2. **Dropping scale metadata from memory estimates.** Groupwise schemes need scales and sometimes zero-points.
3. **Using the wrong axis.** Per-channel along the wrong dimension can change both accuracy and kernel compatibility.
4. **Reusing bad calibration data.** Activation ranges can be workload-dependent.
5. **Assuming storage precision equals compute precision.** Weight-only kernels often compute/accumulate at higher precision.
6. **Ignoring unquantized exceptions.** Norms, embeddings, output heads, or selected layers may remain high precision.
7. **Benchmarking checkpoint size as latency.** Smaller files do not guarantee faster generation.
8. **Forgetting packing/layout.** A mathematically correct quantizer can be unusable by a target kernel.
9. **Mixing incompatible scales/checkpoints.** Quantization metadata is part of the model contract.
10. **Publishing paper speedups as universal.** Hardware, batch, sequence length, and software version must match.

Mental model. Treat a quantized checkpoint as weights + quantization metadata + a required execution contract.

Research anchors: [3]-[16]

38 - 完整的 理解框架（Mental Model）

对应原文第 40 页；点击查看相邻原文对照页

38 - 十个问题中的完整 理解框架（Mental Model） 量化（Quantization） 当您看到 量化后的（Quantized） 模型（Model） 时，问：

1. 量化后的（Quantized） 是什么？权重、激活（Activations）、键值缓存（KV Cache） 或其中几个？

2.什么格式？ INT8、INT4、FP8、NF4、FP4 还是其他？

3.什么映射？均匀仿射 浮点（Floating Point） 还是非均匀 码本（Codebook）？

4.什么粒度？张量（Tensor）、通道、组、词元（Token）、还是块？

5.如何选择尺度？校准（Calibration），运行时amax，重建，学习截断（Clipping）？

6. 异常值会怎样？剪掉、隔离、重新缩放、保护或保持高 精度（Precision）？

7. 低 精度（Precision） 的实际计算是什么？存储和算术可能有所不同。

8. 还保留哪些元数据？标度、零点、密码本、异常、填充？

9. 哪个瓶颈得到改善？容量、带宽、计算、延迟（Latency）、吞吐量（Throughput）？

10. 内核（Kernel）/硬件消耗哪些？没有后端支持，压缩可能不会变得速度。

一旦这些问题明确，GPTQ、AWQ、SmoothQuant、LLM.int8()、NF4、FP8 和 KV 缓存方法就更容易比较。它们并不是对同一问题的相互竞争的答案；而是相互竞争的。它们在设计空间中占据不同的位置。

理解框架（Mental Model）。 量化（Quantization） 是受控数值逼近。有用的问题不是“有多少位？”但是“哪些信息是近似的，使用什么地图，以及该近似可以释放哪些系统优势？”了解 量化（Quantization） 研究锚点：[1]-[17]

38 - THE COMPLETE MENTAL MODEL

Quantization in ten questions

When you see a quantized model, ask:

1. **What is quantized?** Weights, activations, KV cache, or several of them?
2. **What format?** INT8, INT4, FP8, NF4, FP4, something else?
3. **What mapping?** Uniform affine, floating point, or non-uniform codebook?
4. **What granularity?** Tensor, channel, group, token, or block?
5. **How are scales chosen?** Calibration, runtime amax, reconstruction, learned clipping?
6. **What happens to outliers?** Clip, isolate, rescale, protect, or keep high precision?
7. **What actually computes low precision?** Storage and arithmetic may differ.
8. **What metadata remains?** Scales, zero-points, codebooks, exceptions, padding?
9. **Which bottleneck improves?** Capacity, bandwidth, compute, latency, throughput?
10. **Which kernel/hardware consumes it?** Without backend support, compression may not become speed.

GPTQ, AWQ, SmoothQuant, LLM.int8(), NF4, FP8, and KV-cache methods are easier to compare once these questions are explicit. They are not competing answers to one identical problem; they occupy different points in the design space.

Mental model. Quantization is controlled numerical approximation. The useful question is not "how many bits?" but "which information is approximated, with what map, and what system benefit does that approximation unlock?"

Research anchors: [1]-[17]

参考

对应原文第 41 页；点击查看相邻原文对照页

参考文献 主要论文和当前实施来源

1. B. Jacob 等人的 量化（Quantization） 和 训练（Training） 的仅用于高效整数算术的神经网络 推理（Inference）。 CVPR 2018。

2. M. Nagel 等人的神经网络白皮书 量化（Quantization）。 arXiv：2106.08295，2021。

3. T. Dettmers、M. Lewis、Y. Belkada、L. Zettlemoyer.LLM.int8()：8 位 矩阵乘法（Matrix Multiplication）

4. Z. Yao 等人的 ZeroQuant：适用于大型变压器的高效且经济实惠的 训练后量化（Post-training Quantization）。 NeurIPS 2022。arXiv：2206.01861。

5. G.Xiao 等人的SmoothQuant：针对大语言 模型（Models） 的准确高效的 训练后量化（Post-training Quantization）。 ICML 2023。arXiv：2211.10438。

6. E. Frantar、S. Ashkboos、T. Hoefler、D. Alistarh.GPTQ：准确的 训练后量化（Post-training Quantization）

7. J. Lin 等人的 AWQ：用于设备上 LLM 压缩和加速的 激活（Activation） 感知权重 量化（Quantization）。 MLSys 2024。arXiv：2306.00978。

8. T. Dettmers、A. Pagnoni、A. Holtzman、L. Zettlemoyer.QLoRA：量化后的（Quantized） LLM 的高效微调。 NeurIPS 2023。arXiv：2305.14314。

9. P. Micikevicius 等人。深度学习的 FP8 格式。 arXiv：2209.05433，2022。

10. Z. Liu 等人.KIVI：键值缓存（KV Cache） 的免调整非对称 2 位 量化（Quantization）。 ICML 2024。

arXiv：2402.02750。

11. C. Hooper 等人 KVQuant：迈向 1000 万上下文长度 LLM 推理（Inference） 和 键值缓存（KV Cache） 量化（Quantization）。 NeurIPS 2024。arXiv：2401.18079。

12. W. Shao 等人。OmniQuant：针对大语言 模型（Models） 进行全方位校准 量化（Quantization）。 arXiv：2308.13137，2023。

了解 量化（Quantization）

13. PyTorch 和 torchao 文档。 量化（Quantization） 观察者、假 量化（Quantization） 和当前的 量化（Quantization）/QAT API。当前文档查阅于 2026 年 9 月。

14.NVIDIA Transformer 引擎（Transformer Engine）文档。 FP8、MXFP8、NVFP4、缩放（Scaling） 配方，混合精度执行。当前文档查阅于 2026 年 9 月。

15.NVIDIA TensorRT-LLM 文档。 量化（Quantization） FP8/FP4、GPTQ/AWQ 和 键值缓存（KV Cache） 的配方。当前文档查阅于 2026 年 9 月。

16.bitsandbytes 文档和源。 NF4/FP4 码本、4 位 打包（Packing）、QLoRA 集成。

当前消息来源咨询于 2026 年 9 月。

17.D.吉田。 NF4 并不是理论上信息最优的（这很好）。

arXiv：2306.06965，2023。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

REFERENCES

Primary papers and current implementation sources

1. B. Jacob et al. *Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference.* CVPR 2018.
2. M. Nagel et al. *A White Paper on Neural Network Quantization.* arXiv:2106.08295, 2021.
3. T. Dettmers, M. Lewis, Y. Belkada, L. Zettlemoyer. *LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale.* NeurIPS 2022. arXiv:2208.07339.
4. Z. Yao et al. *ZeroQuant: Efficient and Affordable Post-Training Quantization for Large-Scale Transformers.* NeurIPS 2022. arXiv:2206.01861.
5. G. Xiao et al. *SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models.* ICML 2023. arXiv:2211.10438.
6. E. Frantar, S. Ashkboos, T. Hoefler, D. Alistarh. *GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers.* ICLR 2023. arXiv:2210.17323.
7. J. Lin et al. *AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration.* MLSys 2024. arXiv:2306.00978.
8. T. Dettmers, A. Pagnoni, A. Holtzman, L. Zettlemoyer. *QLoRA: Efficient Finetuning of Quantized LLMs.* NeurIPS 2023. arXiv:2305.14314.
9. P. Micikevicius et al. *FP8 Formats for Deep Learning.* arXiv:2209.05433, 2022.
10. Z. Liu et al. *KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache.* ICML 2024. arXiv:2402.02750.
11. C. Hooper et al. *KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization.* NeurIPS 2024. arXiv:2401.18079.
12. W. Shao et al. *OmniQuant: Omnidirectionally Calibrated Quantization for Large Language Models.* arXiv:2308.13137, 2023.
13. PyTorch and torchao documentation. Quantization observers, fake quantization, and current quantization/QAT APIs. Current documentation consulted September 2026.
14. NVIDIA Transformer Engine documentation. FP8, MXFP8, NVFP4, scaling recipes, mixed-precision execution. Current documentation consulted September 2026.
15. NVIDIA TensorRT-LLM documentation. Quantization recipes for FP8/FP4, GPTQ/AWQ, and KV cache. Current documentation consulted September 2026.
16. bitsandbytes documentation and source. NF4/FP4 codebooks, 4-bit packing, QLoRA integration. Current source consulted September 2026.
17. D. Yoshida. NF4 Isn't Information Theoretically Optimal (and that's Good). arXiv:2306.06965, 2023.